

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Grafové databáze – přehled a srovnání



2024

Vedoucí práce:
Mgr. Tomáš Urbanec

Jan Gaja

Studijní program: Informatika,
Specializace: Obecná informatika

Bibliografické údaje

Autor: Jan Gaja
Název práce: Grafové databáze – přehled a srovnání
Typ práce: bakalářská práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2024
Studijní program: Informatika, Specializace: Obecná informatika
Vedoucí práce: Mgr. Tomáš Urbanec
Počet stran: 47
Přílohy: elektronická data v úložišti katedry informatiky
Jazyk práce: český

Bibliographic info

Author: Jan Gaja
Title: Graph databases – overview and comparison
Thesis type: bachelor thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2024
Study program: Computer Science, Specialization: General Computer Science
Supervisor: Mgr. Tomáš Urbanec
Page count: 47
Supplements: electronic data in the storage of department of computer science
Thesis language: Czech

Anotace

Práce má dva cíle: 1. nastudovat principy ukládání a práce s daty v grafových databázích a získané poznatky sepsat do textu, který čtenáři poskytne dobrou představu o fungování, výhodách a nevýhodách tohoto přístupu k datům. 2. vybrat několik implementací grafových databází, nachystat ukázky jejich použití a pokusit se navrhnout experimenty ukazující silné a slabé stránky zvolených implementací.

Synopsis

Thesis has two goals: 1. study the principles of storing and working with data in graph databases and and write down the knowledge gained in a text that will give the reader a good idea of how it works, advantages and disadvantages of this approach to data. 2. select several implementations of graph databases, prepare examples of their use and try to devise experiments showing the strengths and weaknesses of the selected implementation.

Klíčová slova: grafové databáze; implementace grafových databází

Keywords: graph databases; implementation of graph databases

Děkuji, děkuji, děkuji.

Místopřísežně prohlašuji, že jsem celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

datum odevzdání práce

podpis autora

Obsah

1	Základní pojmy z teorie grafů	9
2	Grafové databáze	11
2.1	Ukládání a zpracování dat	12
2.2	Identifikátory	12
2.3	Grafové modely	13
2.3.1	Property graph	13
2.3.2	Resource Description Framework	13
2.3.3	Hypergrafy	13
2.4	Výhody grafových databází	15
2.5	Nevýhody grafových databází	15
2.6	Implementace grafových databází	16
2.6.1	Neo4j	17
2.6.2	OrientDB	17
2.7	Dotazovací jazyky v grafových databázích	17
2.7.1	Cypher	18
2.7.2	Gremlin	18
3	Modelování grafových dat	19
3.1	Definice problému	20
3.1.1	Co od naší aplikace uživatel očekává?	20
3.1.2	Jaké typy informací naše aplikace potřebuje uchovávat, aby měla danou funkcionalitu?	22
3.1.3	Jak spolu tyto pojmy vzájemně souvisí v rámci aplikace?	23
3.1.4	Na jaké otázky potřebují uživatelé najít odpověď?	23
3.2	Konceptuální datový model	24
3.2.1	Entity	24
3.2.2	Vztahy	24
3.2.3	Konceptuální model	25
3.3	Logický datový model	25
3.3.1	Vrcholové labely	26
3.3.2	Hranové labely	28
3.3.3	Vlastnosti	29
3.4	Testování modelu	30
3.4.1	Unit testy	30
3.4.2	Profile a Explain	30
3.4.3	Indexy	31
3.4.4	Denormalizace	32
4	Hledání cest mezi 2 vrcholy	32
4.1	Procházení grafu	33
4.2	Cesta v neohodnoceném grafu	34
4.3	Cesta v ohodnoceném grafu	35

5	Algoritmy	37
5.1	Algoritmy pro hledání cesty v grafu	38
5.1.1	Dijkstra	38
5.1.2	A*	38
5.1.3	Minimální kostra grafu	39
5.1.4	Random walk	39
5.2	Algoritmy pro míru centrality	40
5.2.1	Degree Centrality	41
5.2.2	Closeness Centrality	41
5.2.3	Betweenness Centrality	42
5.2.4	PageRank	42
5.2.5	Použití algoritmů pro míru centrality	42
5.3	Detekce komunit	43
5.3.1	Počítání trojúhelníků	43
5.3.2	Hledání komponent	44
	Závěr	45
	Conclusions	46
	Seznam zkratk	47

Seznam tabulek

1	Tabulka obsahující soubory vygenerovaných dat pro aplikaci pro cestovní nadšence.	22
2	Tabulka obsahující soubory pro ukázkou hledání cest.	33
3	Výsledná tabulka pro dotaz ve zdrojovém kódu 3.	36
4	Výsledná tabulka pro dotaz 4.	38
5	Výsledek algoritmu Random walk s parametrem inOutFactor nastaveným na hodnotu 2 zavolaným na stanice s vlastností stationId menší než 10.	40
6	Výsledek algoritmu Random walk s parametrem inOutFactor nastaveným na hodnotu 0.2 zavolaným na stanice s vlastností stationId menší než 10.	41
7	Algoritmy pro měření centrality aplikované na jednotlivé vrcholy z grafu na obrázku 14.	42
8	Počet trojúhelníků a shlukovací koeficient jednotlivých vrcholů z grafu na obrázku 14.	44

Úvod

Grafové databáze, stejně jako jiné databáze (například relační, NoSQL) představují systém pro uchovávání a zpracovávání dat. Jsou, jak již název napovídá, postaveny na jednom z oborů diskrétní matematiky jménem teorie grafů, podobně jako jsou relační databáze postaveny na pojmu relace. Pracují tedy s daty v podobě grafu. Díky této struktuře mají skvělé vlastnosti pro aplikace, které ve větší míře pracují se vztahy mezi jednotlivými entitami.

První kapitola obsahuje základní pojmy z teorie grafů používané v textu.

Druhá kapitola se věnuje grafovým databázím – co to je, k čemu to je dobré.

Třetí kapitola popisuje modelování dat v grafových databázích.

Čtvrtá kapitola se zabývá cestami v grafu.

V páté kapitole je nahlédnuto na některé z algoritmů využívaných v grafových databázích. Jsou zde přehledově představeny algoritmy pro hledání cest v grafu, algoritmy pro míru centrality a podívali jsme se na problém detekce komunit.

Šestá kapitola pojednává o implementacích grafových databází a dotazovacích jazycích v nich používaných. Převážně pak je pozornost věnována databázi Neo4j a OrientDB.

1 Základní pojmy z teorie grafů

V této části jsou uvedeny základní definice z teorie grafů používané v textu.

Definice 1 (Graf)

Graf G je uspořádaná dvojice $\langle V, E \rangle$, kde V označuje neprázdnou množinu vrcholů a E označuje množinu hran. Každá hrana je určena dvěma vrcholy z V , které spojuje.

V grafu $G = \langle V, E \rangle$ se V nazývá *množina vrcholů* grafu G a označuje se $V(G)$. Podobně E se nazývá *množina hran* grafu G a označuje se $E(G)$.

POZNÁMKA 2 (VRCHOL)

V teorii grafů se pojem vrchol někdy zaměňuje s pojmem uzel. V této práci však budeme používat pojem vrchol.

POZNÁMKA 3 (NEKONEČNÉ GRAFY)

Grafy mohou mít nekonečnou množinu vrcholů. Tyto grafy se nazývají nekonečné. Nekonečné grafy však obecně nelze uchovávat v počítači. V této práci tedy budeme pod pojmem graf myslet graf s konečnou množinou vrcholů.

Grafy se dělí na *orientované* a *neorientované*. Hrana v neorientovaném grafu spojující vrcholy u a v se nazývá *neorientovaná* a zapisuje se pomocí množiny obsahující dvojici vrcholů, tedy $\{u, v\}$. Hrana v orientovaném grafu začínající ve vrcholu u a končící ve vrcholu v se nazývá *orientovaná* a pro zápis se používá uspořádaná dvojice, tedy $\langle u, v \rangle$. O hraně začínající i končící ve stejném vrcholu mluvíme jako o *smyčce*.

Grafy se nejčastěji znázorňují pomocí kroužků, které představují vrcholy, a úseček mezi nimi, sloužící pro reprezentaci hran. Pokud se jedná o orientovaný graf místo úseček používáme šipky. Pro hrany používáme označení *vícenásobné*, pokud existuje mezi dvěma vrcholy hran více. Grafu obsahujícímu vícenásobné hrany se říká *multigraf*.

Orientované i neorientované grafy též mohou mít dodatečnou informaci ve formě *ohodnocení hran*. Obecně ji můžeme definovat jako funkci $w : E \rightarrow \mathbb{R}$. *Ohodnoceným grafem* pak nazveme trojici $G = (V, E, w)$. Hodnotu $w(e)$ pro hranu $e \in E$ ohodnoceného grafu $G = (V, E, w)$ nazýváme *váha* nebo *ohodnocení hrany* e . Obdobně lze definovat ohodnocení vrcholů jako funkci $w : V \rightarrow \mathbb{R}$.

POZNÁMKA 4 (OHODNOCENÝ GRAF)

Ohodnoceným grafem v textu budeme vždy myslet graf $G = (V, E, w)$, kde přiřazení w je ohodnocení hran.

Definice 5 (Symetrizace orientovaného grafu)

Symetrizace orientovaného grafu $G = \langle V, E \rangle$ je neorientovaný graf $G' =$

$\langle V, E' \rangle$, pokud pro každou hranu $\{u, v\} \in E'$ platí, že $\langle u, v \rangle \in E$, nebo $\langle v, u \rangle \in E$.

Symetrizaci grafu lze popsat jako proces, ve kterém zapomeneme orientaci hran, odstraníme smyčky a vícenásobné hrany nahradíme jednoduchými a vznikne nám tak neorientovaný graf.

Definice 6 (Sousedství)

Řekneme, že vrcholy $u, v \in V$ v grafu $G = \langle V, E \rangle$ spolu *sousedí*, pokud existuje mezi nimi hrana $e \in E$. Říkáme, že vrcholy u, v jsou *incidentní* s e . *Sousedství* vrcholu $v \in V$ je množina vrcholů $N(v)$ obsahující všechny vrcholy, se kterými vrchol v sousedí.

Definice 7 (Stupeň vrcholu)

Stupněm vrcholu $u \in V$ v grafu $G = \langle V, E \rangle$ se rozumí počet hran s ním incidentních. Značíme jej $\deg(u)$.

Vrcholy se stupněm 1 se v teorii grafů nazývají *koncové* vrcholy, nebo *listy*. Vrcholy se stupněm 0 se nazývají *izolované*. Stupeň vrcholu u lze v orientovaném grafu dále rozdělit podle hran vcházejících a vycházejících na *vstupní stupeň vrcholu* u definovaný jako $\deg_{in}(u) = |\{\langle v, u \rangle | v \in V\}|$ a *výstupní stupeň vrcholu* u definovaný jako $\deg_{out}(u) = |\{\langle u, v \rangle | v \in V\}|$.

Definice 8 (Podgraf)

Podgraf grafu $G = \langle V, E \rangle$ je graf $H = \langle V_1, E_1 \rangle$, kde $V_1 \subseteq V$ a $E_1 \subseteq E$, tak že každá hrana z E_1 má oba vrcholy ve V_1 .

Neformálně podgraf vznikne z původního grafu odstraněním některých hran, nebo vrcholů spolu s jejich incidentními hranami. Podgraf $H = \langle V_1, E_1 \rangle$ nazveme *indukovaný*, pokud E_1 obsahuje všechny hrany z E , pro které platí, že mají oba konce v množině V_1 .

Definice 9 (Sled)

Sled v grafu $G = \langle V, E \rangle$ je sekvence vrcholů a hran tvaru $v_0, e_1, v_1, e_2, \dots, e_n, v_n$, kde vrchol $v_i \in V$ pro všechna $i \in \{0, 1, 2, \dots, n\}$ a pro hranu e_i platí:

- $e_i = \{v_{i-1}, v_i\} \in E$ pro všechna $i \in \{1, 2, \dots, n\}$, je-li G neorientovaný.
- $e_i = \langle v_{i-1}, v_i \rangle \in E$ pro všechna $i \in \{1, 2, \dots, n\}$, je-li G orientovaný.

Definice 10 (Cesta)

Cesta je sled, kde se vrcholy ani hrany neopakují.

V neohodnocených grafech má cesta obsahující $n + 1$ vrcholů délku n . Délka cesty v ohodnoceném grafu je rovna součtu ohodnocení hran na cestě.

Definice 11 (Kružnice)

Kružnice je cesta, kde počáteční a koncový vrchol je stejný. U orientovaného grafu používáme pojem *cyklus*.

O grafu $G = \langle V, E \rangle$ řekneme, že je *cyklický*, pokud se v grafu vyskytují kružnice, nebo cykly. Pokud v grafu neexistuje kružnice, graf nazveme *acyklický*.

Definice 12 (Vzdálenost)

Pro vrcholy u, v grafu G definujeme *vzdálenost* vrcholu u a vrcholu v definujeme jako délku cesty z u do v , která je ze všech délek cest z u do v nejmenší. Značíme ji $dist(u, v)$.

Definice 13 (Souvislý graf)

Neorientovaný graf $G = \langle V, E \rangle$ nazýváme *souvislý* pokud pro každé dva vrcholy $u, v \in V$ platí, že existuje cesta z u do v . *Komponenta grafu* G je každý jeho maximální souvislý podgraf. U orientovaného grafu $H = \langle V_H, E_H \rangle$ rozlišujeme slabou a silnou souvislost. Graf H je *slabě souvislý*, pokud jeho symetrizace je souvislý graf. O grafu H řekneme, že je *silně souvislý*, pokud pro každé dva vrcholy $u, v \in V_H$ platí, že existuje cesta z u do v .

Definice 14 (Strom)

Strom je souvislý graf $G = \langle V, E \rangle$ neobsahující kružnice. Množina navzájem nepropojených stromů se nazývá *les*.

Definice 15 (Kořenový strom)

Kořenový strom je dvojice $T = \langle G, r \rangle$, kde $G = \langle V, E \rangle$ je strom a $r \in V$. Vrchol r nazýváme *kořen*.

S kořenovými stromy se pojí další pojmy. *Hloubka vrcholu* v ve stromě T s kořenem r je rovna $dist(r, v)$. Speciálně hloubka kořene r je 0. *Výškou kořenového stromu* T rozumíme nejdelší vzdálenost z kořene do některého z vrcholů v T . Vrchol v nazýváme *předchůdce* (rodič) vrcholu u , pokud v kořenovém stromu T existuje cesta z kořene r do u ve tvaru $r \dots, v, e, u$. Vrchol u se pak nazývá *následník* (přímý potomek) v . Pokud vrchol v se vyskytuje na cestě z kořene do u , nazýváme vrchol v *předek* u a vrchol u *potomek* v . Vrcholy mající stejného rodiče označujeme jako *sourozence*. *Listem* nazýváme vrchol, který v kořenovém stromu nemá žádné následníky. Naopak *vnitřní vrchol* je nazýván ten vrchol, který není listem.

2 Grafové databáze

Grafová databáze je specializovaná NoSQL databáze pro uchovávání a dotazování dat, které jsou mezi sebou provázány pomocí vztahů. Jedná se o systém, který rozšiřuje grafový model o operace CREATE, READ, UPDATE a DELETE

(zkráceně CRUD). Grafové modely jsou představeny v samostatné podkapitole nesoucí stejný název. Data v databázi jsou uchovávána ve formě vrcholů a hran, které spolu tvoří graf. Jsou obvykle určeny pro použití s transakčními systémy (OLTP).

2.1 Ukládání a zpracování dat

Grafové databáze můžeme mezi sebou rozlišovat podle zpracování a ukládání dat.

Grafová databáze má *nativní grafové uložště*, pokud umožňuje uchovávání a správu dat v grafové podobě. Tento přístup zefektivňuje následnou práci s daty. Nicméně ne všechny grafové databáze se vydávají při ukládání dat stejnou cestou. Některé z nich tato data serializují do jiných typů databází. Tento způsob se především opírá o vyspělost a známost jiných technologií, hlavně co se týče relačních databází.

Druhou vlastností je *index-free adjacency*. Ta nám říká, že v uložšti jsou s každým vrcholem mimo jiné uloženy přímé odkazy (pointery) na sousední vrcholy. Pro přístup k sousedům každého vrcholu tedy není potřeba žádná další datová struktura, nebo globální index. Obdobně to platí pro hrany. S každou hranou máme též uloženy ukazatele na koncové vrcholy, které jsou s danou hranou incidentní. Výsledkem je skvělý výkon při procházení grafem, protože z aktuálního vrcholu se lze dostat na specifický vrchol s konstantní časovou složitostí. Navíc výkon není závislý na velikosti grafu v databázi, nýbrž na množství procházených dat. Toto je obrovská výhoda ve srovnání s relačními databázemi, kde je zpracování požadavku závislé na velikosti spojovaných tabulek. Nevýhodou tohoto přístupu je, že některé dotazy (zejména ty při kterých nedochází k průchodu grafem) mohou být na výkon nebo paměť mnohem náročnější.

Nativními grafovými databázemi označujeme ty, které s daty manipulují a uchovávají je v grafové podobě. Pracují efektivněji při průchodu grafem. Pro práci s velkým množstvím hustě propojených dat je tato architektura navrhnutá pro zacházení s daty nezbytná. Nenativní grafové databáze jsou bezpečnější volbou pro malé projekty a ne příliš propojená data, neboť obvykle staví na technologiích, které jsou známé. V této práci se budeme dále zabývat pouze nativními grafovými databázemi.

2.2 Identifikátory

Každý vrchol a každá hrana v grafu mají své vlastní jedinečné identifikátory (ID). To je rozdíl oproti relačním databázím, ve kterých sloupec s identifikátory může a nemusí být v tabulce. Tyto identifikátory mohou být uživatelsky měnitelné v některých grafových uložštích, ale častějším případem jsou generovány systémem automaticky.

2.3 Grafové modely

Prostor systémů pro uchovávání grafů lze rozdělit podle grafových datových modelů. Mezi grafové modely řadíme například property graph, Resource Description Framework(RDF) a hypergrafy.

2.3.1 Property graph

Jak z názvu vyplývá, jedná se o model, ve kterém jsou grafy doplněny o vlastnosti. Tyto vlastnosti mohou být přiřazeny jak vrcholům, tak hranám. Vlastnosti jsou reprezentované pomocí dvojic klíč-hodnota. Vrcholům jsou též přiřazeny labely. Pomocí nich lze vrcholy seskupovat. Hrany jsou pojmenované a orientované. U každé hrany tedy jde určit počáteční a koncový vrchol. Jedná se o koncept, který je nejvíce využíván v grafových databázích.

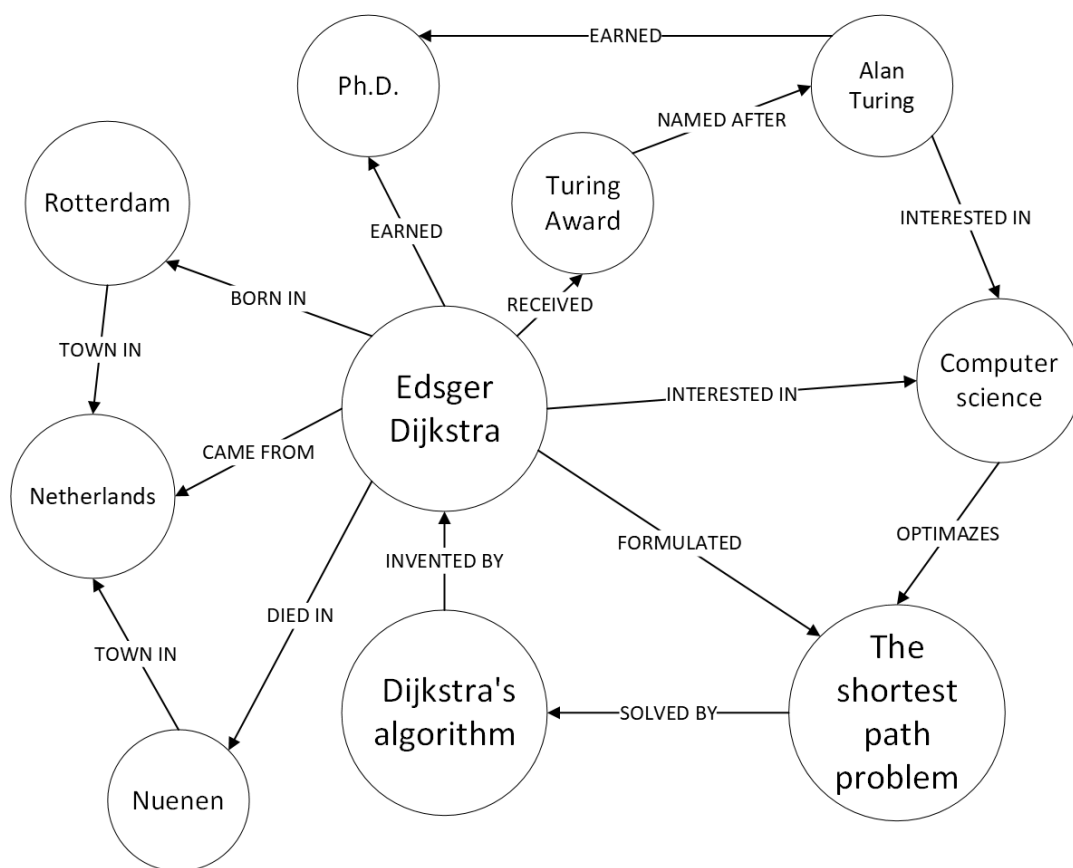
2.3.2 Resource Description Framework

RDF model umožňuje vyjadřovat tvrzení o věcech v podobě jednoduchých vět, které gramaticky sestávají z podmětu, přísudku a předmětu. Tyto věty nazýváme tvrzení. Tvrzení samozřejmě nevyjadřujeme jako věty v přirozeném jazyku, ale v počítačové notaci. RDF model zavádí následující strukturu zvanou trojice: subjekt predikát objekt. Subjekt je obdobou podmětu, predikát je obdobou přísudku a objekt je obdobou předmětu. Každá věta popisuje hranu spojující vrcholy definované subjektem a objektem. Predikát je pak pojmenováním hrany. RDF model je známý ze sémantického webu, kde je použit pro odvození informací pomocí sémantického označení odkazů spojujících webové zdroje. Tento model též též nachází využití v aplikacích využívajících znalostní grafy (anglicky knowledge graphs). Příkladem těchto grafů je například Google Knowledge Graph ¹. Google jej využívá například ve svém hlasovém ovládání Google Asistenta, nebo od května 2012 data získaná z tohoto grafu vizualizuje v tzv. Knowledge Graph panelu. Knowledge graph panel zobrazuje obrázek 1.

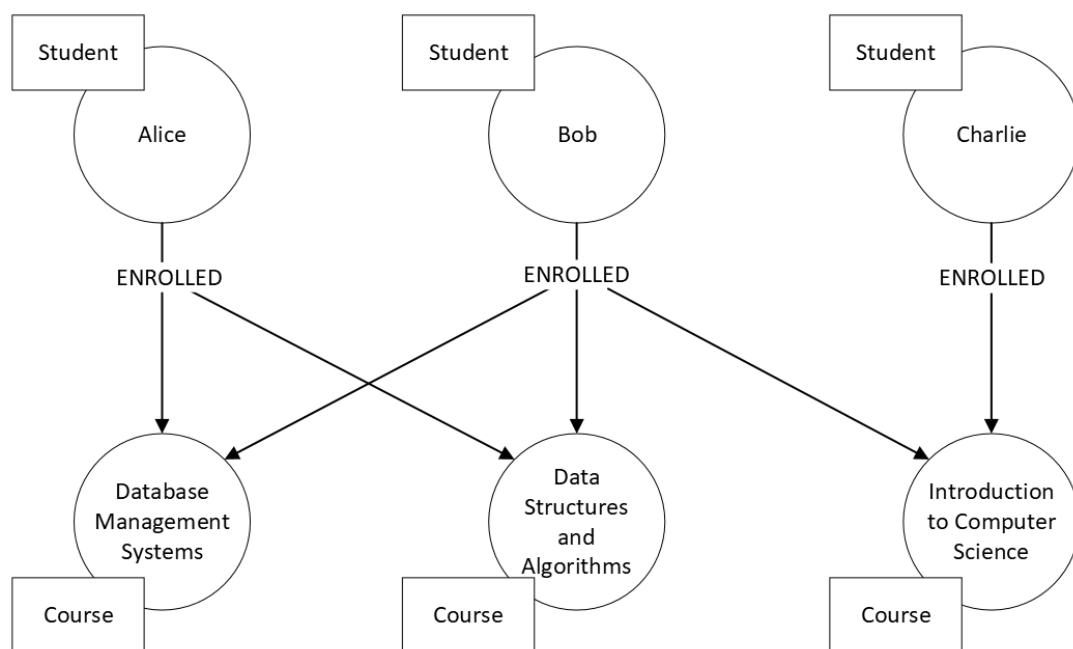
2.3.3 Hypergrafy

Hypergraf vznikl zobecněním klasického grafového modelu. Hypergraf obsahuje speciální hrany zvané hyperhrany, které mohou spojoval více vrcholů. Narodil od klasické orientované hrany, hyperhrana může mít libovolné množství počátečních a koncových vrcholů. Hypergrafy mohou být užitečné v aplikacích, které zacházejí s vazbami o kardinalitě M:N. Vždy je možné reprezentovat informace v hypergrafu jako property graph. I když s použitím více vztahů a zprostředkujících vrcholů. Příklad hypergrafu je ukázán na obrázku 2.

¹<https://blog.google/products/search/introducing-knowledge-graph-things-not/>



Obrázek 1: Knowledge graph pro nizozemského informatika Edsgera Dijkstru.



Obrázek 2: Hypergraf znázorňující studenty zapsané na kurzy.

2.4 Výhody grafových databází

Jednou z velkých výhod těchto systémů je, že uchovávají jak samotné entity, tak všechny vztahy mezi entitami (pomocí index-free adjacency). Tedy při dotazování nedochází k výpočtu těchto vztahů, jak tomu je například u relačních databází pomocí klauzule JOIN. To zrychluje zpracování některých typů dotazů. Jak již bylo řečeno, množství dat obecně výkon nezhoršuje, neboť čas pro vyhodnocení dotazu je úměrný pouze velikosti podgrafu, kterým procházíme. Pro aplikace, které pracují se vztahy mezi entitami ve velké míře, je proto grafová databáze ideálním řešením.

Pomocí vrcholů a hran lze často lépe vystihnout dané problémy, než například pomocí tabulek. Grafy nám poskytují přirozený způsob pro ukládání a manipulaci z daty. Popisují jak nad daty v rámci aplikace přemýšlíme. Lze tak dosáhnout datových modelů, které jsou srozumitelné i laikům, což ve velkých projektech, na kterých pracuje velká spousta lidí, může být obrovský benefit.

Ve spoustě případech je dotazování prostřednictvím grafových dotazovacích jazyků přehlednější než dotazy v jiných databázích. To platí především pro dotazy pracující s relacemi mezi entitami.

Grafové databáze jsou obecně schema-less. Tedy struktura dat se může měnit v závislosti na potřebách aplikace. Přidávání nových vlastností, vrcholů, hran a labelů nenarušuje chod aplikace. Což přináší pozitiva pro vývojáře a budoucí vývoj.

Dotazy v grafových databázích mohou vracet různé typy dat, bez toho aby ztratily výsledky smysl. V relačních databázích lze v jednom dotazu vracet výsledky pro různé entity pomocí klauzule UNION, ale vrácená data potrádají smysl, neboť vrácená tabulka obsahuje tato data různých entit smíchaná dohromady. V grafových databázích toho lze využít například k získání všech dostupných informací ke konkrétnímu zákazníkovi, které máme v databázi, v rámci jednoho dotazu.

Lze je využít k uchovávání vztahů mezi slovy a frázemi. Tedy v kombinaci s počítačovým zpracováním jazyka jsou vhodné v oborech jako data science, nebo machine learning.

I přes problematické shardování nabízí spousta implementací grafových databází horizontální škálování. To nám umožňuje rozdělit data na více výpočetních strojů v případě uchovávání enormních grafů.

2.5 Nevýhody grafových databází

Grafové databáze jsou mezi ostatními druhy databází poněkud mladé. Narozdíl od relačních databází je všechny mezi sebou nepropojuje jeden standardizovaný dotazovací jazyk. Jeho absence škodí jejich popularitě, neboť pro vývojáře znamená, že pro práci s více grafovými databázemi je nucen používat více různých dotazovacích jazyků, místo specializování se na jeden univerzální. Navíc při změně grafové databáze hrozí, že bude nutné přepsat všechny dotazy do jiného dotazovacího jazyka. Spousta databází používá speciálně upravený jazyk SQL,

umožňující základní práci s databází. Jednou z nich je OrientDB. Práce s databází je však při použití SQL značně omezená. Avšak nejednotnost dotazovacího jazyka by mohla být v blízké době minulostí. Podle dostupných informací se pracuje na normovaném jazyku GQL, což je zkratka pro Graph Query Language².

Data, která mají grafovou strukturu též nejsou dobrou volbou pro aplikace využívající transakce. Při velkém objemu transakčních dat nejsou tak efektivní jako jiné.

Grafovým databázím též dělají problém dotazy, které operují nad celým grafem. Jedná se především o filtrování a agregování. Pokud se v aplikaci soustředíme především na takové typy operací je lepší zvolit jiné databázové technologie.

Jelikož jsou grafové databáze často schema-less může docházet k nekonzistenci ukládaných dat. Ne všechny implementace grafových databází totiž disponují mechanismy, které by v případě potřeby nekonzistenci zabránily. Konzistence dat je tedy často řešena pouze na aplikační úrovni.

2.6 Implementace grafových databází

Existuje spousta implementací grafových databází. Liší se například nativností; dotazovacím jazykem, škálovatelností a podporou ACID transakcí. Jakou z nich využít pro naši aplikaci je otázka preferencí. My se zaměříme na dvě implementace, a to Neo4j a OrientDB. Dalšími známými databázemi jsou například:

- Amazon Neptune³ – rychlý, spolehlivý, plně řízený systém pro hustě propojené grafy. Výhodou je podpora známých dotazovacích jazyků Gremlin, openCypher a SPARQL.
- TigerGraph⁴ – moderní, nativní, paralelní grafová databáze pro ukládání property graphs. Na webu uvádějí až dvacetčtyřikrát rychlejší načítání dat oproti jiným implementacím.
- ArangoDB⁵ – nabízí nativní, multi-model databázový systém. Data mohou být uložena jako páry klíč-hodnota, dokumenty, nebo grafy a ke všem lze přistupovat pomocí jednotného dotazovacího jazyka AQL.
- FlockDB – vyvinuta společností Twitter (teď X?). Jedná se o nenativní grafovou databázi, navrženou pro real-time sociální aplikace.
- JanusGraph⁶ – jedná se o open-source projekt společnosti The Linux Foundation. Je založen na Apache TinkerPop a tedy lze klást dotazy prostřednictvím jazyka Gremlin.

²<https://www.gqlstandards.org>

³<https://aws.amazon.com/neptune>

⁴<https://www.tigergraph.com>

⁵<https://arangodb.com>

⁶<https://janusgraph.org>

2.6.1 Neo4j

Databázový systém Neo4j je nejrozšířenější open-source grafovou databází, nabízející vysoký výkon při práci s grafy. Její první verze byla vyvinuta v roce 2002 a vznikla z důvodu nespokojenosti jejích zakladatelů s výkonem RDBMS. Implementována je v Javě. Jedná se o nativní grafovou databázi. Je využívána hromadou světoznámých společností jako je například Microsoft, ebay, nebo LinkedIn. Klást dotazy můžeme prostřednictvím jazyka Cypher.

Neo4j nabízí velké množství způsobů jak interagovat s grafovými daty. Každý z nich je určen pro jinou cílovou skupinu. Například v této práci je využita aplikace Neo4j Desktop, která slouží pro správu lokálních instancí Neo4j. Verze používané aplikace je 1.5.9. Dalším produktem je cloudový databázový servis pojmenovaný Neo4j AuraDB. Vznikl v roce 2019. Je také dostupný v bezplatné verzi, která je však omezená – lze v ní spravovat nejvýše 200 tisíc vrcholů a 400 tisíc hran. Neo4j má též spoustu rozšiřujících knihoven a nástrojů, které lze do produktů doinstalovat (pokud již nejsou součástí). Například se jedná o nástroj pomáhající při migrování dat z relační databáze do Neo4j, standardní knihovna procedur a funkcí apod. V práci je pak ukázána knihovna gds s užitečnými grafovými algoritmy.

2.6.2 OrientDB

OrientDB je open-source NoSQL databázový systém implementovaný v Javě. Byl představen v roce 2010. Je to pokročilá multi-model databáze - podporuje grafový, dokumentový, klíč-hodnota a objektový model. Manipulovat s databází můžeme pomocí programovacího jazyka Java, speciálně upraveného SQL a dotazovacího jazyka Gremlin. Uživatelům je pak umožněno interagovat s databází skrze aplikaci, API, konzolu nebo prostřednictvím grafického uživatelského rozhraní – OrientDB Studio. Pokud OrientDB Server běží lokálně, OrientDB Studio lze najít ve webovém prohlížeči na adrese <http://localhost:2480>. OrientDB je schématicky flexibilní. Umožňuje uchovávat schema-full, schema-less a schema-hybrid data.

Nejmenší jednotkou v databázi OrientDB je záznam (record). Rozlišujeme 4 typy záznamů: Documents, Blobs, Vertices a Edges. Pro každý záznam pak je automaticky vygenerováno ID záznamu, označeno jako RID (Record ID). RID je uchováván ve formátu `#<cluster>:<position>`.

- `<cluster>` označuje identifikátor clusteru, do kterého záznam patří.
- `<position>` udává absolutní pozici záznamu v rámci clusteru.

V práci je využívána verze 3.2.23.

2.7 Dotazovací jazyky v grafových databázích

Jak již bylo zmíněno pro grafové databáze zatím neexistuje jeden standardní jazyk. Lze narazit na jazyky vyvinuté specificky pro konkrétní systém. Příkladem

může být jazyk *Cypher* vyvíjený společností Neo4j. Na druhou stranu jsou zde jazyky jakým je například *Gremlin*, který je více univerzálnější a lze jej použít ve více systémech zacházejících s grafy. Mezi další známé jazyky patří:

- *SPARQL* – využíváný pro znalostní grafy v RDF formátu
- *PGQL* – je jednou z komponent datové databáze společnosti Oracle, který je postavený na SQL
- *GSQL* – procedurální jazyk používaný v databázi TigerGraph podobný SQL
- *AQL* – dotazovací jazyk pro manipulaci s daty v databázi ArangoDB podobný SQL
- *GraphQL* – jedná se o dotazovací jazyk, který není závislý na konkrétní implementaci. Za jeho vývojem stojí Facebook. Je považován za přímého konkurenta REST API.

2.7.1 Cypher

Cypher je inspirovaný známým jazykem SQL z relačních databází. Důkazem může být použití několika stejnojmenných klíčových slov, jako je WHERE, LIMIT, ORDER BY apod. Jeho myšlenkou je definování, jaká data chceme z grafu dostat, místo definování jak je dostat. Cypher je unikátní v tom, že využívá kulaté závorky pro vrcholy a hranaté pro hrany, čímž docílí lepší přehlednosti v dotazu. Základní syntaxe pro průchod grafem z vrcholu *vrchol1* na vrchol *vrchol2*, přes hranu HRANA, která je spojuje, je ve formátu *(vrchol1)-[:HRANA]->(vrchol2)*.

Obdobně jako v SQL je dotaz v Cypheru složen ze tří základních částí. První je povinná a začíná klíčovým slovem MATCH. Určuje, které vrcholy a hrany se mají vybrat. Následuje část začínající klíčovým slovem WHERE. Ta je nepovinná. Filtruje vybrané elementy z první části dotazu na základě jejich vlastností. Poslední, povinná, část začíná klíčovým slovem RETURN. Definuje, co je po provedení dotazu vráceno. Na výsledky lze aplikovat další operace, jako jsou například agregování, limitování, nebo seřazování.

2.7.2 Gremlin

Gremlin je dotazovací jazyk pro procházení grafů, který je součástí grafového výpočetního frameworku Apache TinkerPop⁷. Gremlin lze použít pro všechna grafová uložiska, která tento framework podporují. Jedná se o imperativní jazyk s nějakými deklarativními prvky.

Apache TinkerPop mimo jiné obsahuje také Gremlin Console, což je textové **REPL** rozhraní pro práci s lokálními, či vzdálenými grafy. Je založená na Groovy konzoli. Groovy je programovací jazyk, který je označován jako nadmnožina

⁷<http://tinkerpop.apache.org/>

Javy. Co funguje v Javě funguje i v Groovy. V konzoli tedy můžeme používat validní kód napsaný v tomto jazyce. Díky tomu lze například uložit do proměnné určitý vrchol a na něj se pak přes danou proměnnou odkazovat. Konzole je široce rozšiřitelná pomocí různých pluginů. Pro napojení na databázi OrientDB budeme potřebovat plugin *tinkerpop.orientdb*. Když je tento plugin aktivovaný, lze v konzoli vytvořit novou instanci *OrientGraph*. Uložíme ji do proměnné *graph*. Než začneme psát dotazy pomocí Gremlinu musíme ještě vytvořit instanci zdrojového objektu procházení grafu zavoláním metody *traversal()* na hodnotu v proměnné *graph*. Celý postup je uveden ve zdrojovém kódu 1.

```
1 //OrientDb database connector
2 graph = OrientGraph.open(url, user, password);
3
4 //establishing a graph traversal source object
5 g = graph.traversal();
```

Zdrojový kód 1: Ukázkový kód v dotazovacím jazyku Gremlin.

3 Modelování grafových dat

POZNÁMKA 16 (PŘEKLAD)

V práci je použito české slovo „vlastnosti“ přeložené z anglického slova „properties“, se kterým se běžně uživatel setká při práci s grafovou databází. Slovo „label“ je z důvodu znělosti zachováno v původním znění.

Modelování dat je proces, který překládá entity reálného světa a vztahy mezi nimi do jejich softwarové reprezentace. Uživatel popisuje doménu jako graf skládající se z vrcholů a hran pomocí vlastností a labelů. Každý model je unikátní. Podoba výsledného modelu vždy závisí na případech použití a otázkách, které mají podobu dotazů, které budou na databázi kladeny. Dodržení základních principů modelování dat je důležité pro výslednou efektivitu. Když je graf přizpůsoben dotazům, dokáže velmi rychle nalézt potřebnou odpověď. Naopak špatně zvolený model grafu může ve výsledku aplikaci ublížit a být překážkou. Například tím, že zpracování dotazu bude trvat déle než u optimalizovaného modelu. Další důležitou věcí, na kterou je potřeba myslet, je budoucnost vyvíjené aplikace. Model by měl být připravený na změny, které po aplikaci budou její koncoví uživatelé požadovat. Správnému modelu je proto dobré věnovat dostatečné množství času a dodržovat obecná pravidla při jeho vytváření.

Je potřeba také podotknout, že ne všechna data jsou vhodná pro uchovávání v grafové podobě. Rozhodnutí, zda pro naše účely je grafová databáze dobrým řešením, je ve spoustě případů náročné. Existují různé typy databází (například relační, dokumentové, nebo klíč-hodnota), kde každá má své silné a slabé stránky a je potřeba si dobře promyslet, kterou zvolit. Lze dohledat spoustu „slovních

návodů“ pomáhajících s touto volbou. Na obrázku 3 je ukázán rozhodovací strom z knihy Graph Databases IN ACTION. Grafové databáze jsou výhodné pro aplikace, které mají vztahy mezi entitami na prvním místě. Tato vlastnost by měla být odrazovým můstkem při rozhodování, zda použít, či nepoužít grafovou databázi.

Pokud se pro požití grafové databáze rozhodneme, můžeme přistoupit k modelování dat. V této kapitole se tedy zaměříme na návrh grafového modelu. Zkusíme si to na příkladu sociální platformy pro nadšence cestování. Pracovně ji nazveme Travel Enthusiast Network. Sociální sítě jsou totiž jedním z případů, kdy použití grafových databází dává smysl. Tato aplikace tedy bude umožňovat cestovatelům se propojit s ostatními. Dále bude zaznamenávat jejich výlety do různých destinací, hodnocení těchto cest, cestovatelské aktivity, které destinace nabízí a které se mohou uživatelům aplikace líbit. Na základě svých přátel a hodnocení cest do destinací aplikace dokáže uživateli doporučit jiné destinace, které dosud nenavštívil. Naše aplikace se tedy bude dotýkat i dalšího případu užití grafových databází, a to systémů pro doporučování, které jsou v praxi hojně využívány. Pro vyzkoušení dotazů můžeme využít csv souborů, obsahujících vygenerovaná data pro naši aplikaci. Seznam všech souborů, rozdělený na entity a vztahy mezi nimi a počet řádků (včetně hlavičky) jednotlivých souborů lze vyčíst z tabulky 1.

Postup modelování se dá rozdělit do čtyř fází:

1. Definice problému
2. Konceptuální datový model
3. Logický datový model
4. Testování modelu

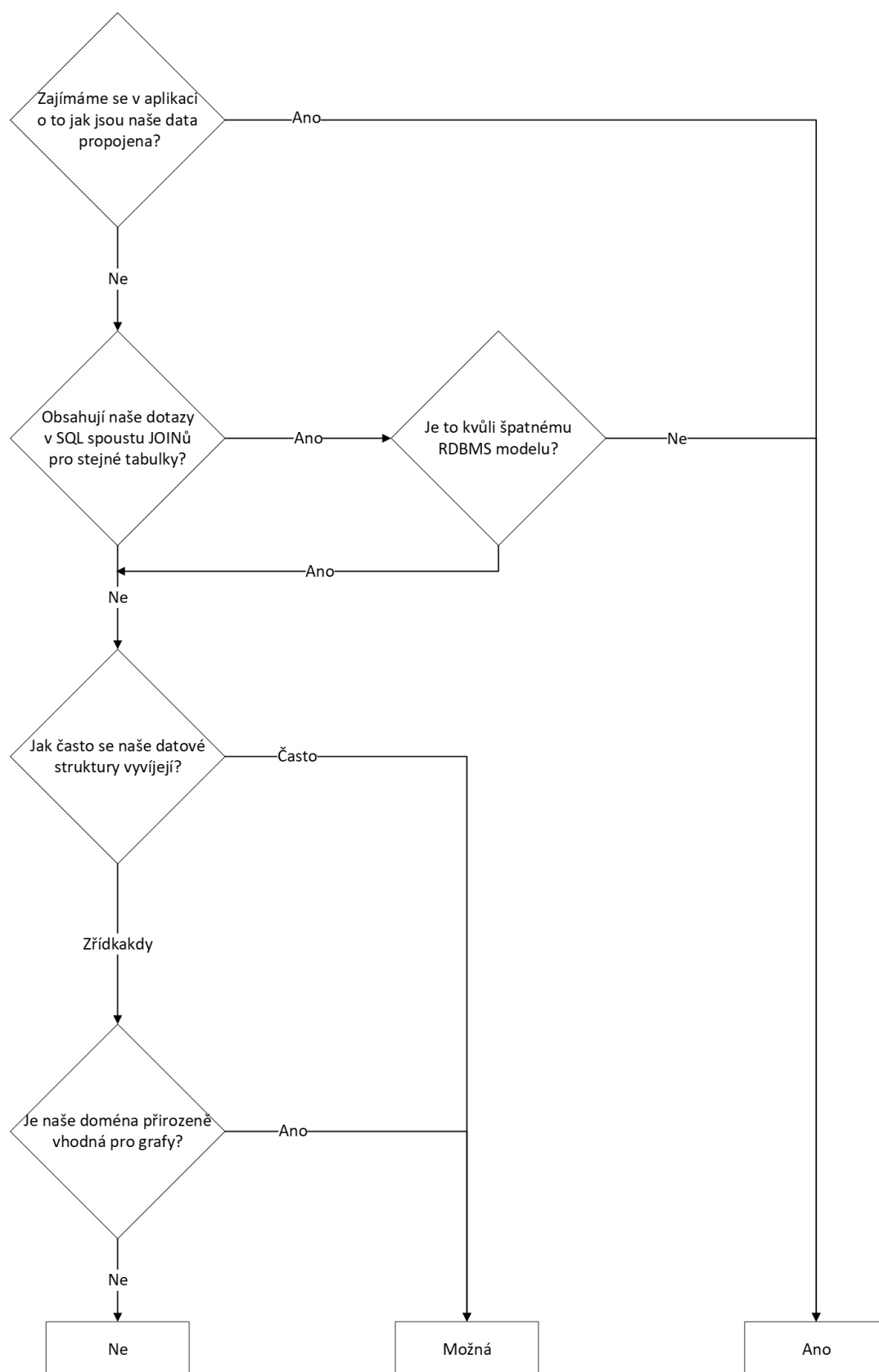
V následujících částech se na jednotlivé fáze podíváme detailněji.

3.1 Definice problému

V první fázi bychom měli zvážit co všechno by měla aplikace umět. Výsledkem by měl být její, co nejpřesnější, slovní popis. Pomoci nám může těchto pár otázek.

3.1.1 Co od naší aplikace uživatel očekává?

Aplikace je sociální síť, proto by měla jistě umožnit se propojit s ostatními uživateli aplikace. Uživatelům by na základě jejich přátel, oblíbených aktivit a hodnocení destinací měly přicházet tipy na výlet. Aplikace by měla umožnit zaznamenat, že někteří uživatelé podnikli cestu společně.



Obrázek 3: Rozhodovací strom pro otázku: Je pro naši aplikaci grafová databáze vhodná?

Typ	Název souboru	Počet řádků	Informace o souboru
Entity	activities.csv	49	Aktivity, které mohou destinace nabízet.
	countries.csv	58	Státy, v kterých se vyskytují jednotlivé destinace.
	destinations.csv	204	Destinace, které mohou uživatelé navštívit.
	journeys.csv	297	Cesty, v rámci kterých uživatelé navštěvují destinace.
	ratings.csv	641	Hodnocení jednotlivých cest uživateli.
	users.csv	301	Uživatelé sociální sítě Travel Enthusiast Network.
Vztahy	follows.csv	5738	Vztah mezi uživateli.
	is_within.csv	204	Vztah mezi destinacemi a státy.
	led_to.csv	1442	Vztah mezi cestami a destinacemi.
	likes.csv	1517	Vztah mezi uživateli a cestovními aktivitami.
	lives_in.csv	301	Vztah mezi uživateli a destinacemi.
	offers.csv	2023	Vztah mezi destinacemi a cestovními aktivitami.
	participated.csv	1518	Vztah mezi uživateli a jejich cestami.
	wrote_about.csv	641	Vztahy mezi uživateli, recenzemi a cestami.

Tabulka 1: Tabulka obsahující soubory vygenerovaných dat pro aplikaci pro cestovní nadšence.

3.1.2 Jaké typy informací naše aplikace potřebuje uchovávat, aby měla danou funkcionalitu?

- Všechny základní informace o **uživateli**, aby se s nimi ostatní mohli spojit - jejich jméno, příjmení, unikátní identifikátor, oblíbené cestovatelské aktivity.
- Informace o jednotlivých **destinacích** - jejich lokace, název, unikátní identifikátor, cestovatelské aktivity v rámci destinace.
- Uživatelé je umožněno psát **hodnocení** jednotlivých **cest**, které podniknul.
- Chtěli bychom také umožnit **seskupení uživatelů**, když jedou do dané destinace společně.

V reálném scénáři by projekt pravděpodobně zahrnoval mnoho dalších atributů, ale pro tuto ukázkovou aplikaci to ponecháme omezené. Pro zjednodušení se vyhneme také přesným souřadnicovým informacím a vystačíme si s informací v rámci jakého státu se daná destinace nachází.

Aplikace by tedy měla zacházet s pojmy jako jsou uživatel, destinace, hodnocení, cestovní aktivita.

3.1.3 Jak spolu tyto pojmy vzájemně souvisí v rámci aplikace?

- Uživatel **sleduje** jiného uživatele.
- Uživatel **se zúčastnil** cesty **do** destinace.
- Uživateli se **líbí** cestovatelská aktivita.
- Uživatel **napsal** recenzi **o** cestě.
- Destinace **nabízí** cestovatelskou aktivitu.
- Destinace **je ve** státě.

3.1.4 Na jaké otázky potřebují uživatelé najít odpověď?

- Jaké všechny uživatele sleduji? A koho sledují oni? Které uživatele nesleduji ale sledují je uživatelé, které sleduji?
- Najdi 3 nejlépe hodnocené destinace, které jsem nenavštívil, nabízející konkrétní cestovní aktivity?
- Které destinace navštívil uživatel X? Kolikrát je navštívil?
- Najdi mi všechny uživatele v rámci skupiny, s kterou cestuje uživatel X do destinace Y.
- Najdi X destinací nabízející některé z cestovatelských aktivit, které se uživateli líbí.
- Pro konkrétní destinaci vrať měsíc v roce v kterém je nejlépe hodnocená.
- V kterých destinacích a před kolika lety jsem byl v tento konkrétní den a měsíc?

V reálném scénáři by otázek mohlo být mnohem více.

3.2 Konceptuální datový model

Dalším krokem je vyvinout konceptuální model. Výstupem by měl být přehledný diagram popisující důmyslně promyšlené schéma naší aplikace z bussiness pohledu. Vyvarujeme se v něm pro nevyvojáře zbytečným detailům a zaměříme se pouze na to, jak data různých typů v rámci vyvíjené aplikace spolu souvisí. Při vyvíjení datového modelu vývojáři často kreslí na tabuli, jak jsou data v rámci projektu propojena s dalšími daty, proto se tato fáze v různých literaturách nazývá whiteboard model. Tento proces vyžaduje identifikování a seskupení všech entit a vztahů, které se v naší aplikaci vyskytují.

3.2.1 Entity

Začneme entitami. Entity většinou představují objekty, jako například uživatel, účet, destinace, které jsou v databázi uloženy a propojeny vzájemnými vztahy. Je třeba si rozmyslet, které entity lze seskupit. Pokud například naše aplikace obsahuje uživatele, zaměstnance a administrátory je užitečné tuto skupinu entit sloučit do jedné a pojmenovat ji například Osoba. Jedním z osvědčených postupů, jak z našich poznámek z minulé fáze vytáhnout entity, je dívat se po podstatných jménech. Pokud se budeme řídit tohoto postupu, tak z věty „Uživateli se líbí cestovatelská aktivita.“ získáme dvě entity, a to Uživatel a Cestovatelská aktivita.

V naší aplikaci můžeme dohromady rozpoznat 6 různých entit:

- Uživatel
- Cesta
- Destinace
- Aktivita
- Stát
- Recenze

Seskupení uživatelů cestujících spolu do destinací jsme sloučili s entitou Cesta. Jedná se o mírné zjednodušení, které nebude mít na funkčnost žádný velký vliv.

3.2.2 Vztahy

Nyní následuje propojení entit pomocí vztahů. Entity v rámci naší aplikace jsme identifikovali zejména zaměřením se na podstatná jména z poznámek o aplikaci. Vztahy můžeme získat obdobně pomocí sloves. Zde bychom však měli být velmi opatrní. Ve slovese se často ukrývá podstatné jméno. V budoucím vývoji aplikace by pak mohlo dojít k problémům při rozšiřování funkcionality aplikace. Dobrým příkladem je sloveso „recenzovat“. Pokud bychom při modelování zvolili větu „Uživatel recenzoval X.“ místo „Uživatel napsal recenzi o X.“, tak bychom

mohli v budoucím vývoji aplikace narazit na problém, že nemáme entitu představující recenzi, na kterou bychom mohli odkazovat. Snadno si jde domyslet komentování nebo jiný způsob reagování na recenze. Podobně pokud bychom v naší aplikaci upřednostnili větu "Uživatel navštívil destinaci." před větou „Uživatel podnikl cestu do destinace.“, uživatel by pak nemohl danou cestu hodnotit tak, jak to máme v popisu naší aplikace. Takové chyby obecně vedou v budoucnosti k migrování databáze, což je velmi obtížný a zdlouhavý proces přesouvání dat z jedné databáze do druhé, kterému je lepší se vyhnout.

Hledáme tedy slovesa, které představují relaci mezi námi nalezenými entitami. Po aplikaci tohoto doporučení na větu „Uživateli se líbí cestovatelská aktivita.“ získáme vztah se líbí mezi entitami Uživatel a Cestovatelská aktivita. Stejným způsobem postupujeme dál a hledáme všechny důležité vztahy. Může se stát, že při důkladném hledání vztahů nalezneme další entity, které jsme předtím nenalezli. Můžeme se dostat do situace, kdy váháme, zda určitý typ dat má tvořit entitu nebo relaci v našem modelu. V této situaci je obecně doporučeno se rozhodovat podle toho, zda některý dotaz na databázi dostane na vstupu informaci ve formě tohoto typu dat. Pokud ano, tato data bychom měli reprezentovat jako entitu. Například lze takto odvodit, že slovo Uživatel musí být entita, protože je vstupem (proměnnou) v dotazu odpovídajícím na otázku „Které destinace navštívil uživatel X? Kolikrát je navštívil?“.

3.2.3 Konceptuální model

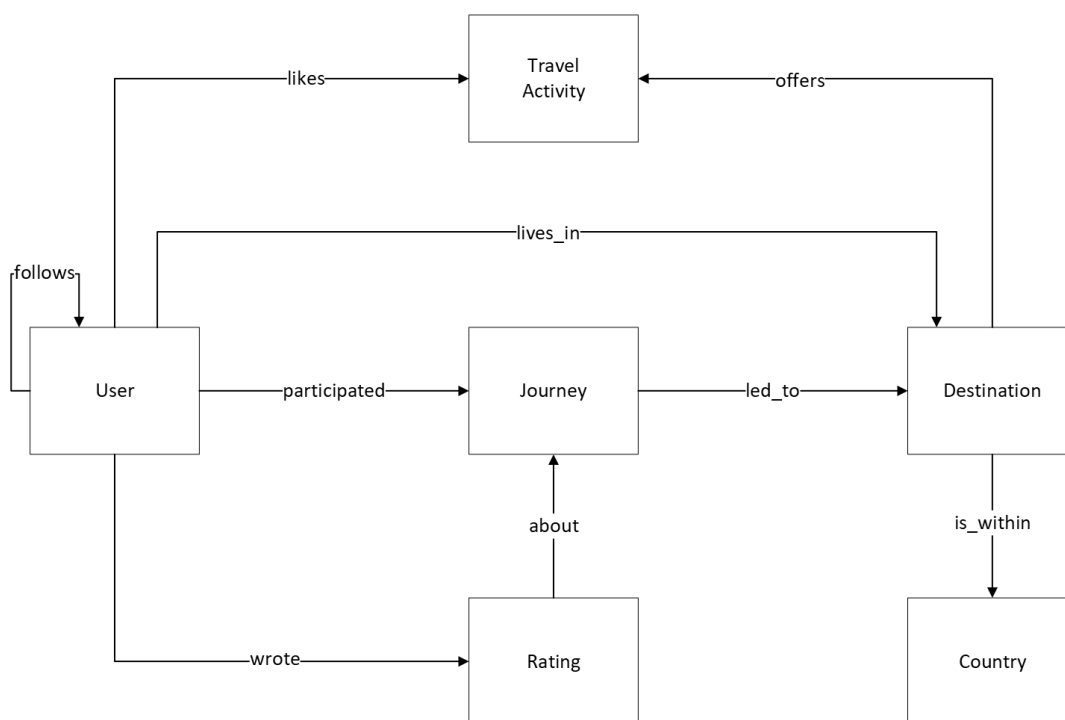
Po důkladném rozmyšlení má naše tabule stejný obsah, jako na obrázku 4.

3.3 Logický datový model

Pokud jsme s diagramem z předchozí fáze spokojeni, můžeme konečně přistoupit ke kroku, ve kterém definujeme základní schéma, které bude určovat podobu našich dat v databázi. V případě relačního modelování je konceptuální model přeformátován, aby mohl být ukládán do tabulek. V případě grafového modelování toto není potřeba. Stačí identifikovat vrcholy a hrany a přiřadit jim příslušný label a vlastnosti. Výstupem tedy bude opět diagram, který ale bude mnohem detailnější. Bude se zabývat detaily, kterými se zabývají vývojáři. Bude definovat tvar, ve kterém budou data v databázi uložena a podle něj budeme moci klást na databázi dotazy. Poté již můžeme do naší databáze vložit zkušební data a vyzkoušet dotazy, které budou odpovídat na naše otázky načrtnuté v první části modelování dat.

Pro vytvoření grafového modelu jsou potřeba tyto 3 kroky:

1. Převést entity na vrcholové labely
2. Převést relace na labely hran
3. Najít a přiřadit vlastnosti vrcholovým a hranovým labelům

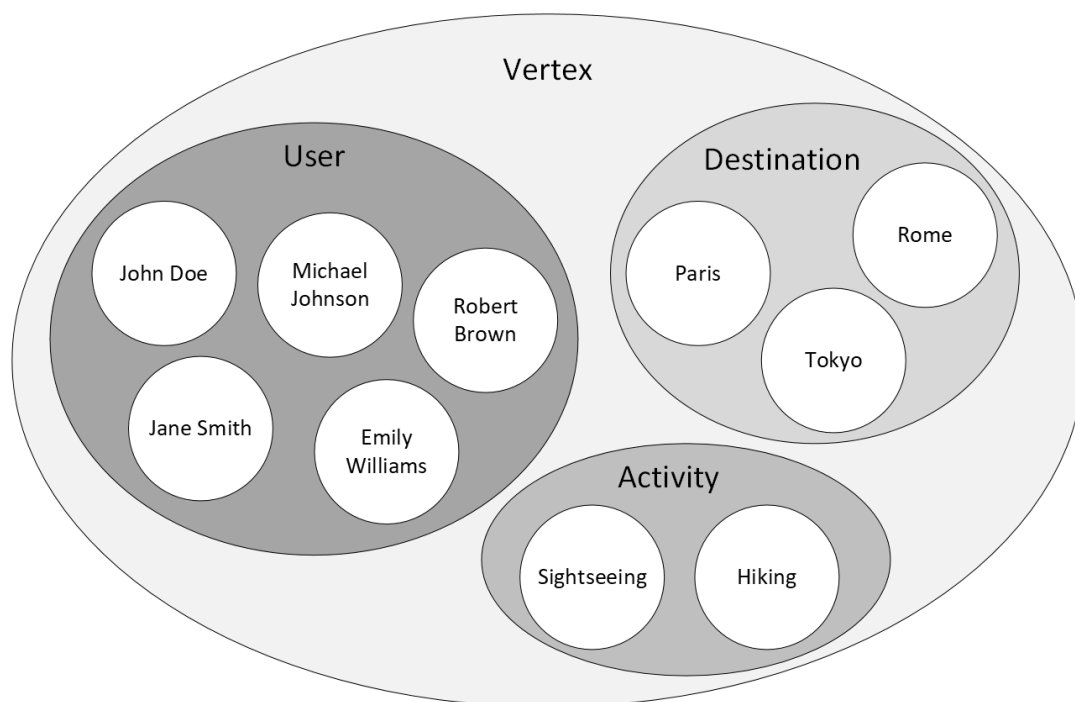


Obrázek 4: Konceptuální datový model pro sociální síť pro cestovní nadšence.

3.3.1 Vrcholové labely

Podstatnou část práce již máme za sebou, neboť můžeme využít dobře navrženého konceptuálního modelu a namapovat naše entity na vrcholové labely. Label vrcholu u určuje, do které skupiny, nebo kategorie vrcholů vrchol u patří. Všechny vrcholy se stejným labelem pak spadají do stejné kategorie. Label pak můžeme používat v dotazech na databázi a pracovat tak s různými skupinami vrcholů a dosáhnout tak lepšího výkonu při dotazování. Pro lepší pochopení je zde obrázek 5. Vymyšlení vrcholových labelů, tedy názvů jednotlivých skupin vrcholů, může být poněkud náročný úkol. Labely by měly být výstižné, krátké a přesné. Musíme mít na paměti budoucí rozšiřitelnost a fakt, že s databází může pracovat jiný vývojář. Dobře zvolený label je tedy velmi žádaný.

Dobrým zvykem je zvolit label, který bude v jednotném čísle. Při přiřazování labelu je dobré myslet na jeho co největší obecnost, protože v budoucnu bychom mohli potřebovat reprezentovat jiný podobný koncept a tedy je užitečné volit co nejobecnější label. Povolený počet labelů pro jeden vrchol se u různých grafových databází liší. Bezpečným přístupem je každému vrcholu přiřadit jeden label. To je například přístup Apache TinkerPop. Často se ale také hodí mít labelů více a pomocí nich umět vyjádřit dědičnost. Vrcholy pak spadají do více skupin vrcholů naráz. Příkladem může být systém pro správu zaměstnanců a klientů v nějaké firmě. Zde by se mohlo hodit kromě labelu Zaměstnanec a Klient mít label Osoba. Neboť zaměstnanec, reprezentovaný vrcholem v databázi, jistě sdílí s klientem informace jako je jméno, datum narození apodobně,

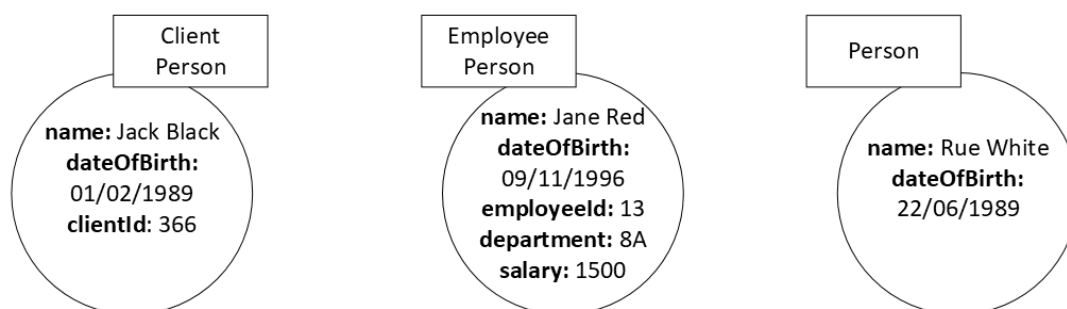


Obrázek 5: Label vrcholu definuje, do které kategorie vrchol spadá.

ale může mít nějaké dodatečné informace, jako je například plat, nebo identifikační číslo zaměstnance. Nyní jednoduše můžeme vybrat data týkající se pouze klientů, ale zároveň můžeme požádat databázi o informaci o všech lidech spravovaných v databázi. Příklad přiřazení více labelů jednomu vrcholu je znázorněn na obrázku 6. To je důvod, proč databáze, jako Neo4j nebo Amazon Neptune umožňují vrcholům přiřadit žádný, jeden, ale i více labelů.

Častou chybou při navrhování labelů je nekonzistentní pojmenovávání konvence. Proto je dobré si nyní stanovit pojmenovací konvenci, kterou se budeme řídit.

POZNÁMKA 17 (KONVENCE POJMENOVÁNÍ)



Obrázek 6: Ukázka více labelů pro jeden vrchol.

V našem příkladu je zvolena konvence doporučená v dokumentaci databáze Neo4j a tedy pro labely vrcholů budeme používat UpperCamelCase notaci, pro labely hran UPPER_CASE notaci s použitím podtržítka pro oddělení slov (SCREAMING_SNAKE_CASE) a pro vlastnosti camelCase notaci.

3.3.2 Hranové labely

Při hledání hran opět využijeme připraveného konceptuálního modelu. Podobně jako u vrcholů se zamyslíme nad labelem hran, které by opět mělo být obecné a výstižné. Navíc bychom se měli držet jedné dobré zásady. Přečtením labelu počátečního vrcholu, labelu hrany z něj vycházející a labelu cílového vrcholu do nějž hrana vstupuje by nám mělo dávat vcelku smysluplnou větu. V češtině však musíme opomenout skloňování. Příkladem by mohla být věta „Uživatel napsal hodnocení.“, která vznikla spojením vrcholového labelu Uživatel, hranového labelu napsal a labelu cílového vrcholu Hodnocení. Ve většině případů mají věty tvar podle následující posloupnosti větných členů podmět-přísudek-předmět.

To nás přivádí k orientaci hran. Předchozí rada nám rovnou specifikuje i orientaci hrany. Tedy z věty „Uživatel napsal hodnocení.“ máme, že hrana s labelem napsal vede z vrcholu s labelem Uživatel do vrcholu s labelem Hodnocení, nikdy ne naopak. Obrácením hrany bychom přečtením vrchol-hrana-vrchol dostali větu „Hodnocení napsalo uživatele.“. Pokud bychom požadovali opačnou orientaci hran mezi vrcholy s labely Uživatel a Hodnocení, bychom museli přistoupit ke změně labelu hrany, neboť by věta postrádala smysl. Tedy například hraně s labelem napsal bychom změnili label na bylo napsáno a pak by věta „Hodnocení bylo napsáno uživatelem.“ dávala smysl. Orientace hrany by měla reflektovat to, jak přirozeným způsobem přemýšlíme o datech v naší doméně.

Než přejdeme k vlastnostem, zbývá ještě určit *jedinečnost* hran. Jedinečnost lze definovat jako počet hran se stejným labelem mezi dvěma vrcholy. Tyto vrcholy mohou a nemusí mít stejný label.

Jedinečnost hran se dělí na dva typy:

- *jednoduchá*,
- *mnohonásobná*.

Jednoduchá jedinečnost odkazuje na žádnou, nebo právě jednu hranu zatímco vícenásobná povoluje hran se stejným labelem mezi dvěma vrcholy více. Na jednoduchou jedinečnost hrany e lze též nahlížet tak, že skupina sousedních vrcholů, do kterých se dostaneme přes hranu e , neobsahuje více instancí jednoho vrcholu, tedy tvoří množinu vrcholů. Zatímco u mnohonásobné jedinečnosti dostaneme kolekci sousedních vrcholů, kde se vrcholy mohou opakovat. Co se týká jedinečnosti hran v naší aplikaci tak všechny hrany, až na hranu mezi Cestou a Destinací budou mít jedinečnost jednoduchou. Například vztah mezi uživatelem a cestovní aktivitou s labelem LIKES buď je nebo není a to lze vyjádřit pomocí existence jedné hrany mezi těmito dvěma vrcholy. Totéž platí pro ostatní hranové labely až na hranu s labelem LED_TO. Jistě totiž může existovat situace,

kdy v rámci cesty uživatel do dané destinace zavítá vícekrát a s touto možností by náš model měl také počítat. Tedy tato hrana bude mít vícenásobnou jedinečnost. Tyto hrany pak od sebe rozlišíme pomocí vlastností, které budou určovat období, kdy se daný uživatel v dané destinaci nacházel. Hrany s mnohonásobnou jedinečností bychom měli rozlišovat pomocí vlastností, aby byly unikátní, jinak tato jedinečnost postrádá smysl.

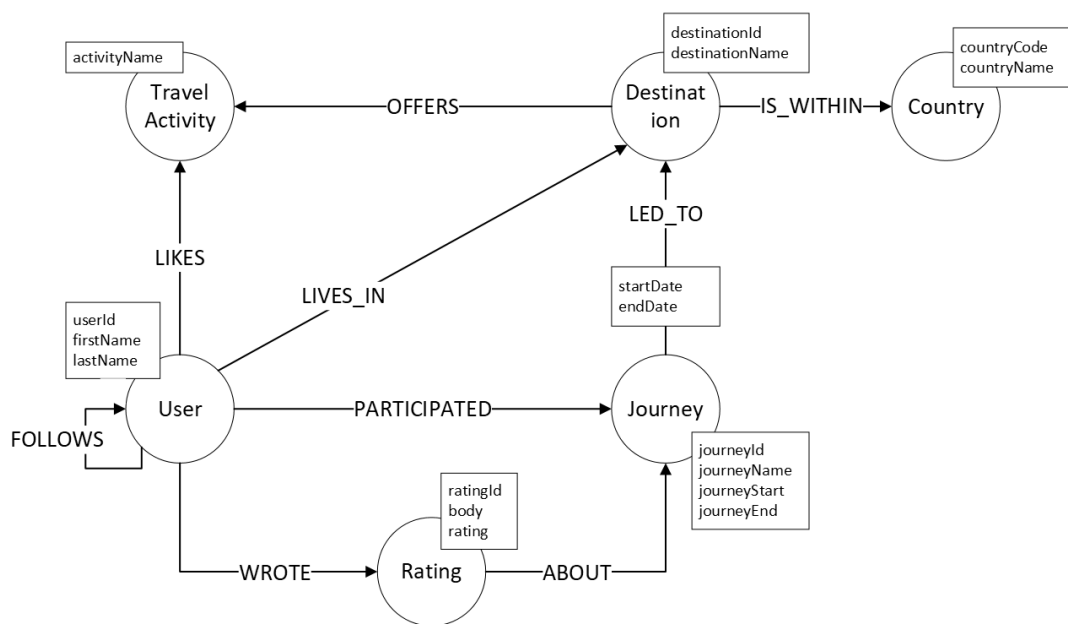
Jak zjistíme, že jsme zvolili špatnou jedinečnost hran? Špatná volba se projeví v dotazech jedním z těchto způsobů:

- Databáze vrací příliš málo dat. Tato chyba nastane pokud zvolíme jednoduchou jedinečnost hran tam, kde by měla být zvolena vícenásobná jedinečnost.
- Databáze vrací duplikativní data. Tato chyba nastane pokud zvolíme vícenásobnou jedinečnost hran tam, kde by měla být zvolena jednoduchá jedinečnost.
- Slabý výkon dotazů na databázi. Tato chyba nastane pokud zvolíme vícenásobnou jedinečnost hran tam, kde by měla být zvolena jednoduchá jedinečnost. Každá hrana stejného labelu mezi dvěma vrcholy navíc znamená rozvětvení v případě průchodu grafem a zvyšuje tedy výslednou časovou složitost dotazu.

U některých implementací grafových databází, jako například DataStax Graph nebo JanusGraph, lze tuto jedinečnost vynutit jako součást při definování schématu. U implementací bez schématického přístupu, jako je Neo4j, tuto jedinečnost lze ošetřit pouze v rámci aplikace.

3.3.3 Vlastnosti

Poslední zastávkou při konstruování logického modelu jsou vlastnosti. Základní kameny grafu vrcholy a hrany máme, ale chybí nám způsob jak rozlišit jednotlivé vrcholy se stejným labelem. K tomu nám slouží vlastnosti. V grafových databázích mohou mít vlastnosti jak hrany tak vrcholy, neboť hrany jsou pro nás stejně významné jak vrcholy. Vlastnosti v grafové databázi jsou páry klíč-hodnota a popisují konkrétní atributy hrany, či vrcholu. Hodnota vlastnosti může být různého datového typu, například řetězec, číslo, pravdivostní hodnota nebo datový typ pro uchování datumu a času. Někdy též může být hodnota typu pole nebo seznam. Pomocí vlastností následně můžeme rozlišovat jednotlivé uživatele například pomocí jejich křestního jména, příjmení, uživatelské přezdívky. To se nám hodí při filtrování dat, když chceme vybrat pouze vrcholy s určitou vlastností. Jednou z vlastností bývá unikátní identifikátor každého vrcholu, díky kterým přistupujeme ke konkrétním vrcholům. K dokončení logického modelu nám tedy zbývá najít a správně přiřadit potřebné vlastnosti konkrétním vrcholovým a hranovým labelům, rozumně je pojmenovat a určit jejich datový typ. K jejich nalezení nám opět mohou pomoci otázky, které chceme aby nám databáze zodpověděla. Z nich můžeme jednotlivé vlastnosti identifikovat a přiřadit.



Obrázek 7: Logický datový model pro sociální síť pro cestovní nadšence.

Naše výsledné grafové schéma popisuje obrázek 7.

3.4 Testování modelu

Testování našeho grafového modelu spočívá ve vyzkoušení, zda se chová dle našeho očekávání (databáze vrací očekávané výsledky) a dotazování není moc pomalé ani pro velké množství dat. K testování lze využít široké škály nástrojů, které dotazovací jazyky a databáze nabízejí.

3.4.1 Unit testy

Při testování správnosti dotazů je vhodné použít tzv. *unit testy*. Tedy vytvoříme si pro každý dotaz malé množství testovacích dat (například kolem 10 vrcholů, tak abychom z něj pouhým okem vyčetli správnou odpověď), kterými otestujeme správnost odpovědí z databáze. Příkladem unit testu je zdrojový kód 2. Tento kód vloží do databáze data, která tvoří graf znázorněný na obrázku 8. Následně zkontroluje dotaz hledající všechny uživatele, které mají od uživatele s vlastností `userId` rovnou 1 vzdálenost rovnou 2. Vidíme, že po spuštění testu databáze by měla vrátit uživatele s identifikátory 3 a 4. Následně po sobě uklidíme a testovací data smažeme.

3.4.2 Profile a Explain

Pokud čelíme pomalému vykonávání některých dotazů, můžeme použít zejména dva nástroje pro diagnostiku problémů. První z nich se nazývá *explain* a slouží pro

```

1 // Vytvoření testovacích dat
2 CREATE (u1:User {userId: 1, firstName: "John", lastName: "Doe"})
3 CREATE (u2:User {userId: 2, firstName: "Alice", lastName: "Smith"})
4 CREATE (u3:User {userId: 3, firstName: "Bob", lastName: "John"})
5 CREATE (u4:User {userId: 4, firstName: "Jane", lastName: "Doe"})
6 CREATE (u5:User {userId: 5, firstName: "Charlie", lastName: "Brown"}
7     )
8
9 CREATE (u1)-[:FOLLOWS]->(u2)
10 CREATE (u2)-[:FOLLOWS]->(u3)
11 CREATE (u1)-[:FOLLOWS]->(u3)
12 CREATE (u2)-[:FOLLOWS]->(u4)
13 CREATE (u3)-[:FOLLOWS]->(u5)
14
15 // Spuštění testu
16 MATCH (u:User {userId: 1})-[:FOLLOWS]->(u2)-[:FOLLOWS]->(u3)
17 WHERE NOT EXISTS ((u)-[:FOLLOWS]->(u3)) AND u3 <> u
18 RETURN u3;
19
20 // Smazání testovacích dat
21 MATCH (u:User) WHERE u.userId IN [1, 2, 3, 4, 5]
22 DETACH DELETE u;

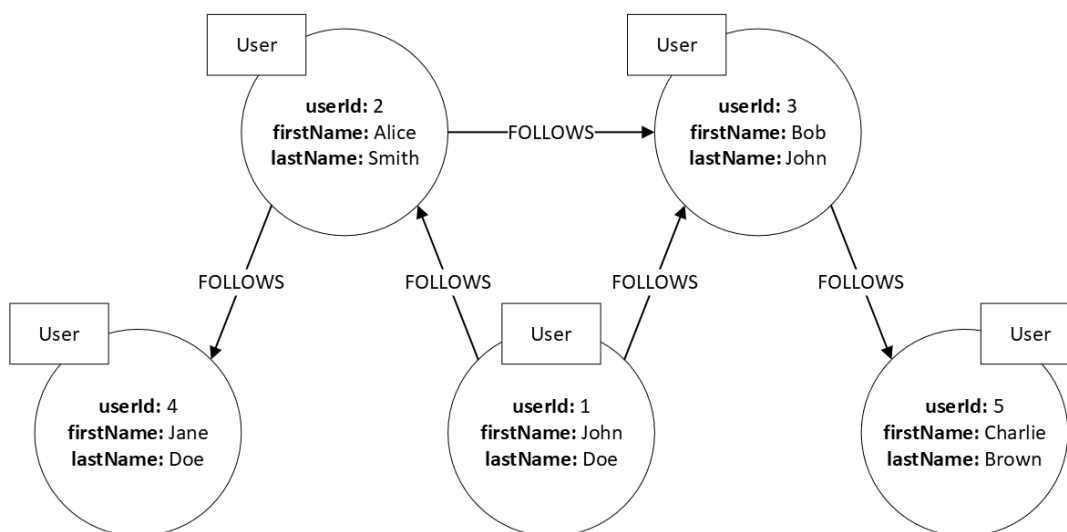
```

Zdrojový kód 2: Ukázkový kód pro kontrolu správnosti dotazu.

vysvětlení jak se bude průchod grafem vyvíjet na základě našeho dotazu. Používá se v situacích, kdy chceme zjistit chování průchodu grafem, ale nechceme jej vykonat. Výsledkem je optimalizovaný exekuční plán daného dotazu, který bude vykonán na datech. Druhému říkáme *profile* a podle dodaného dotazu profiluje, co průchod grafem udělal. Profile je hojně používaným nástrojem pro analýzu dotazů, neboť dotazy jsou často pomalé pouze pro několik vrcholů – například těch, které mají vysoký stupeň. Toho lze využít k porovnání chování rychlých a pomalých vykonání dotazu. Získané informace pak využijeme k optimalizování našich dotazů.

3.4.3 Indexy

Pro efektivní hledání dat v rámci grafu lze použít *indexy*. Toho lze využít pro vylepšení dotazů, neboť díky nim můžeme zrychlit průchod grafem, přístup ke konkrétním datům a další operace pracující s daty. Indexy vytvoří redundantní kopii vybraných dat v databázi podle zadaných kritérií. Jsou výhodné pro data, které jsou častým terčem hledání. Nejčastěji se jedná o vlastnosti, které často vystupují ve filtrování dat v dotazu, nebo identifikátory vrcholů. Dále se využívají pro vlastnosti, na kterých jsou prováděny různé operace s řetězci. Příkladem je hledání slov začínající, končící, nebo obsahující určitý podřetězec. Implementace jako je TinkerGraph poskytuje pouze globální indexy jedné hodnoty. Oproti tomu databáze jako Neo4j nabízí celou škálu různých typů indexů. Indexy také však



Obrázek 8: Graf vytvořený pomocí zdrojového kódu 2.

mají nevýhody. Zvyšují nároky na úložný prostor a mohou zpomalovat zápisy do databáze. Měly by tedy být vytvářeny pouze pro data, pro které budou obzvláště výhodné.

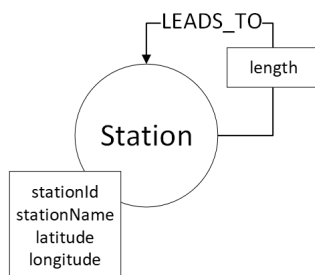
3.4.4 Denormalizace

Denormalizace je metoda, která do databáze přidává redundantní data za účelem zrychlení čtení dat. Denormalizace v grafových databázích většinou znamená kopírování hodnot vlastností vrcholů na incidentní hrany. Často se jedná o časové informace. Nevýhodou tohoto procesu je udržba těchto kopií což vede k zpomalení zápisů. Denormalizace je tedy vhodná pouze pro aplikace, kde počet operací pro čtení z databáze mnohonásobně převyšuje počet operací pro zápis.

4 Hledání cest mezi 2 vrcholy

V této kapitole si odhalíme další velkou výhodu grafových databází a to hledání cest mezi vrcholy. V první části si připomeneme dva základní algoritmy pro průchod grafem a následně této znalosti využijeme ve druhé části při zjišťování dosažitelnosti a hledání nejkratších cest ve smyslu počtu hran mezi dvěma vrcholy. Ve třetí části se podíváme na hledání cesty s nejmenším možným součtem ohodnocení hran na této cestě.

Pro ukázkové příklady je použitý dataset obsahující informace o belgických železničních stanicích a spojeních mezi nimi. Jednoduchý model znázorňuje obrázek 9. V databázi tedy budeme uchovávat tedy pouze vrcholy s labelem `Station` a hrany mezi nimi s labelem `LEADS_TO`. Původní dataset je rozdělen do dvou csv souborů. Jeden uchovává informace o železničních stanicích a druhý spojení



Obrázek 9: Model grafové databáze pro ukázkové příklady v této kapitole.

mezi nimi. Informace obsahující souřadnice pro jednotlivá spojení byly vypuštěny. Seznam souborů shrnuje tabulka 2.

Typ	Název souboru	Počet řádků	Informace o souboru
Entity	stations.csv	560	Informace o železničních stanicích.
Vztahy	leads_to.csv	1386	Vztah mezi stanicemi.

Tabulka 2: Tabulka obsahující soubory pro ukázkou hledání cest.

4.1 Procházení grafu

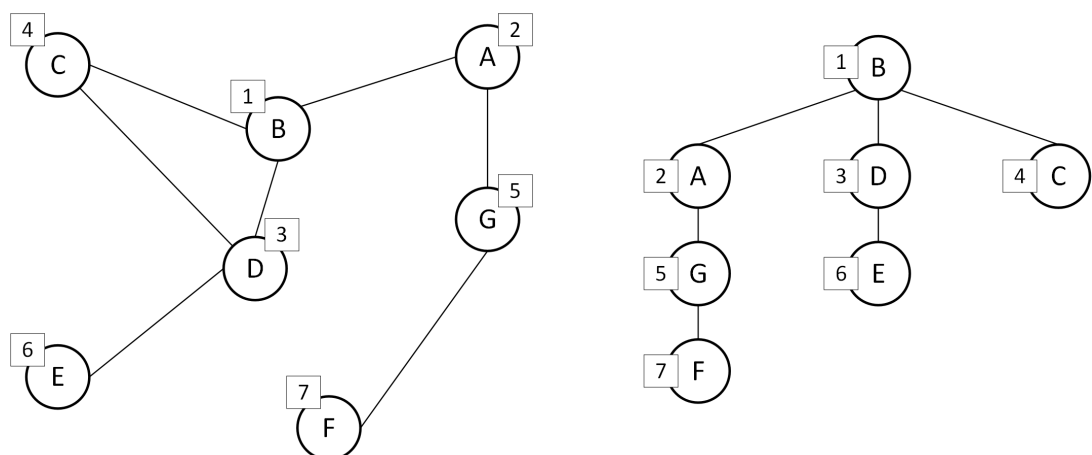
Pro prohledávání grafů od určitého počátečního vrcholu lze použít dva známé grafové algoritmy:

1. Prohledávání do šířky (zkráceně BFS z anglického spojení Breadth-first search)
2. Prohledávání do hloubky (zkráceně DFS z anglického spojení Depth-first search)

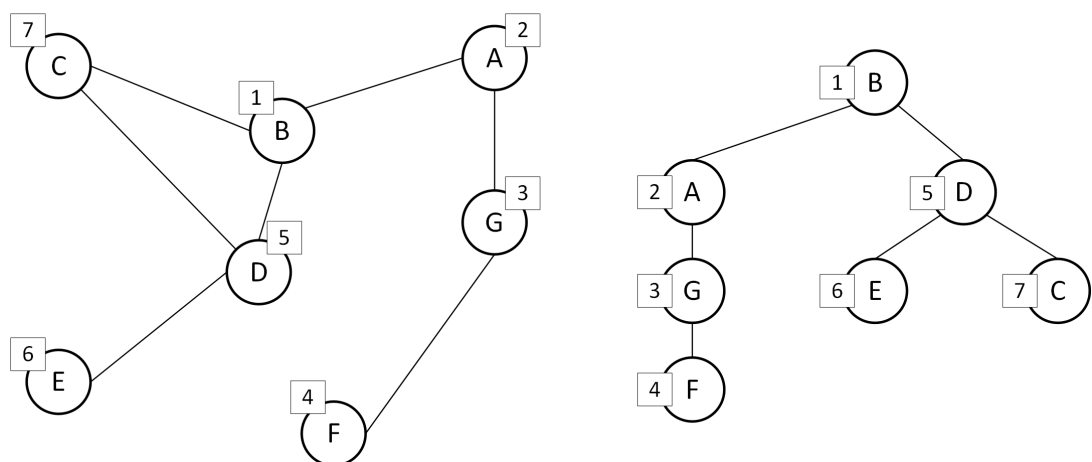
Oba algoritmy mají složitost $O(|E| + |V|)$.

Algoritmus BFS začne v počátečním vrcholu a následně prohledá všechny své sousední vrcholy. Poté projde sousedy sousedů počátečního vrcholu, které ještě nenavštívil a tak postupuje dokud nenavštíví všechny dosažitelné vrcholy. Tento způsob procházení grafu lze znázornit pomocí kořenového stromu. Kořenem je počáteční uzel. Algoritmus prochází vrcholy grafu po jednotlivých úrovních tohoto stromu. Platí, že nejkratší cesta z počátečního vrcholu do vrcholu v v grafu je rovna hloubce v ve stromu, který vznikne průchodem do šířky. Lze jej efektivně implementovat pomocí fronty vrcholů. Průchod grafem do šířky a strom, vzniklý průchodem algoritmu jsou znázorněny na obrázku 10. Pořadí, v kterém jsou vrcholy procházeny je znázorněno čísly v obdélnících u každého vrcholu.

Algoritmus DFS postupuje opačně, v aktuálně navštíveném vrcholu přejde do dosud nenavštíveného souseda, pokud takový soused existuje, jinak provede



Obrázek 10: Průchod algoritmu BFS grafem.



Obrázek 11: Průchod algoritmu DFS grafem.

backtracking – vrací se zpět po cestě odkud přišel, dokud nenalezne vrchol jehož soused nebyl navštíven. Skončí jakmile jsou všechny vrcholy navštíveny. Tento způsob procházení grafu lze opět znázornit pomocí kořenového stromu, kde kořen je počáteční uzel. Avšak v tomto stromu pozorování ohledně nejkratších cest, jaké jsme udělali u BFS obecně neplatí. Při implementaci využíváme zásobník. Průchod grafem do hloubky a strom, vzniklý průchodem algoritmu jsou znázorněny na obrázku 11. Pořadí, v kterém jsou vrcholy procházeny je znázorněno čísly v obdélnících u každého vrcholu.

4.2 Cesta v neohodnoceném grafu

Na tento typ cesty lze nahlížet jako na cestu v grafu, kde každá hrana má stejnou váhu. Tyto cesty lze hledat i v grafu s ohodnocenými hranami tak, že ohodnocení hran ignorujeme. Počet hran na cestě pak určuje délku cesty. Avšak cest mezi

dvěma vrcholy může být obrovské množství. Pokud neomezíme průchod grafem, například pomocí hledání cesty fixní délky, nebo maximální možným počtem hran na cestě, může se stát, že výpočet bude trvat velmi dlouho, nebo v horším případě bude nekonečný. Druhá možnost nastane pokud nezajistíme, že v grafu hledáme opravdu jen cesty a nikoliv sledy. (Cesta i sled byly definovány v první kapitole.) Výpočet by se pak zacyklil a místo výsledku bychom dostali run-time-error.

Tento problém lze vyřešit pomocí algoritmů na prohledávání grafů. Konkrétně se řeší pomocí prohledávání do šířky, neboť systematicky prochází všechny vrcholy, které mají nejkratší vzdálenost od počátečního vrcholu rovnou k , kde $k \in \mathbb{N}_0$. Začíná pro $k = 0$ a postupně k inkrementuje dokud nenarazí na cílový vrchol. S tímto přístupem je zřejmé, že pokud je cílový vrchol dosažitelný ze začínajícího vrcholu, pak v neohodnoceném grafu nalezneme nejkratší cestu. Pro libovolné $k \in \mathbb{N}$ můžeme takto postupovat i pro nalezení k různých nejkratších cest pro dva dané vstupní vrcholy. V případě algoritmu procházení do hloubky bychom nalezení nejkratší cesty obecně dosáhnout nemuseli, i když by řešení mohlo být rychlejší, pokud by se algoritmus vydal správným směrem. To lze opět usoudit z toho, jak je algoritmus implementován. Tedy tento algoritmus je možné využít k zjištění dosažitelnosti vrcholu, což může být v určitých případech výhodné, pokud jsou vrcholy od sebe hodně vzdálené.

V Cypheru lze cesty obsahující 1 až 4 hrany najít pomocí dotazu ve zdrojovém kódu 3. Na řádku 2 vyfiltrujeme ty sledy, které obsahují cyklus. Pro lepší přehled výsledky oindexujeme a seřadíme podle délky cesty. Tabulka 3 ukazuje výsledek tohoto dotazu po zavolání. Obrázek 12 pak znázorňuje podgraf, který obsahuje všechny vrcholy a hrany vystupující v dotazu. Tučnou čarou je zde znázorněna cesta, která ve výsledné tabulce má index 3. Čárkovaně je vyznačena cesta označená v tabulce indexem 1.

```

1 MATCH path=(src:Station {stationId: 157})-[:LEADS_TO*1..4]->(dst:
   Station {stationId: 1061})
2 WHERE ALL(node IN nodes(path) WHERE 1 = size([n IN nodes(path) WHERE
   n = node]))
3 WITH path, [node IN nodes(path) | node.stationName] AS stations
4 ORDER BY size(stations)
5 WITH COLLECT(stations) AS allStations
6 UNWIND range(0, size(allStations) - 1) AS index
7 RETURN index, allStations[index] AS stations;
```

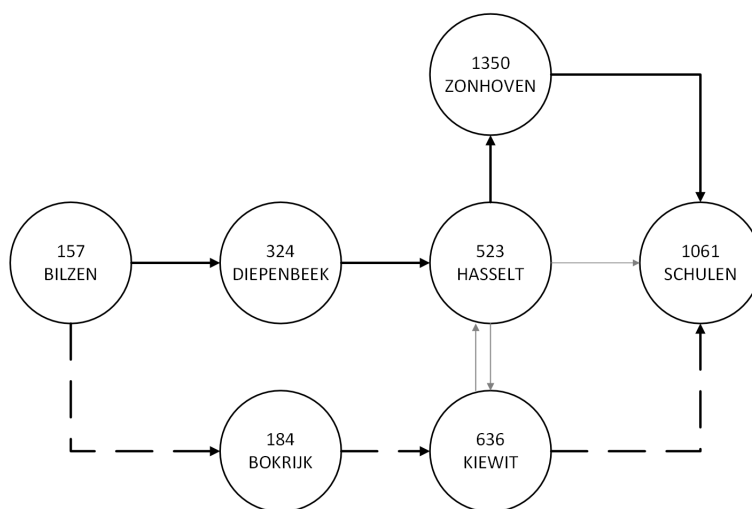
Zdrojový kód 3: Ukázkový kód v Cypheru.

4.3 Cesta v ohodnoceném grafu

Délka cesty mezi počátečním a koncovým vrcholem v ohodnoceném grafu je určena součtem ohodnocení hran na cestě. Tedy nejmenší počet hran na cestě

Index	Stations
0	BILZEN, DIEPENBEEK, HASSELT, SCHULEN
1	BILZEN, BOKRIJK, KIEWIT, SCHULEN
2	BILZEN, DIEPENBEEK, HASSELT, KIEWIT, SCHULEN
3	BILZEN, DIEPENBEEK, HASSELT, ZONHOVEN, SCHULEN
4	BILZEN, BOKRIJK, KIEWIT, HASSELT, SCHULEN

Tabulka 3: Výsledná tabulka pro dotaz ve zdrojovém kódu 3.



Obrázek 12: Podgraf reprezentující výsledek dotazu ve zdrojovém kódu 3.

nutně neznamená, že součet ohodnocení hran je menší než na jiné cestě obsahující více hran. Z tohoto důvodu nemůžeme použít lehce modifikovaný algoritmus, který nám řešil problém v předchozí části. Zde též platí, že cest může být mezi dvěma vrcholy opravdu mnoho. Obzvláště ve velkých grafech. Z těchto důvodů Neo4j nabízí knihovnu optimalizovaných algoritmů, která hledání nejkratší cesty v ohodnoceném grafu nabízí. Na tuto knihovnu se podíváme v další kapitole.

Často se z časových důvodů nejkratší cesta aproximuje. Pokud má vrchol na možné cestě výstupní stupeň větší než nějaký limit l (například $\deg_{out}(u) > 10000$), cestu přes něj nepočítáme, neboť bychom museli procházet každou výstupní hranu což by bylo výpočetně náročné. Těmto vrcholům se říká super-uzly. Více o nich si řekneme v následující kapitole.

Výpočet lze optimalizovat zadáním horního limitu pro délku cesty. Pokud délka cesty tento limit přesáhne, tak tato cesta je zavrhnuta. Další optimalizaci si lze vypůjčit od Dijkstrova algoritmu, a to tu že si budeme udržovat a aktualizovat nejkratší dosud nalezenou cestu do navštívených vrcholů. Tato optimalizace je využita ve zdrojovém kódu 4 v jazyku Gremlin. Výsledek, který dostaneme shrnuje tabulka 4.

Z vrácených výsledků vidíme, že nejkratší délka železnic mezi stanicemi SCHAEER-

```

1  g.withSack(0.0).
2    V().
3    has("Station", "stationId", 1048).
4    repeat(outE("LEADS_TO").
5      sack(sum).by("length").
6      inV().as("visited").
7      simplePath().
8      group("minDist").
9      by().by(sack().min()).
10     filter(project("currMinDist", "sackDist").
11       by(select("minDist").select(select("visited"))).
12       by(sack()).
13       where("currMinDist", eq("sackDist")))).
14    until(has("Station", "stationId", 810)).
15    order().
16    by(sack(), asc).
17    limit(3).
18    project("stations", "stationsCount", "totalLength").
19    by(path().
20      unfold().
21      hasLabel("Station").
22      values("stationName").
23      fold()).
24    by(path().count(local).math('(_+1)/2')).
25    by(sack()).
26    fold()

```

Zdrojový kód 4: Ukázkový kód v dotazovacím jazyku Gremlin.

BEEK a MECHELEN má délku 18.2 kilometrů. Na této cestě projdeme přes 6 stanic. Jen o 50 metrů je delší druhá cesta, která obsahuje o jednu zastávku méně. Třetí výsledek je delší o více než 1.5 kilometru a jedná se o přímý spoj.

5 Algoritmy

V této kapitole si ukážeme řadu užitečných algoritmů, které jsou určeny pro práci s grafem a jsou tedy v určitých situacích vhodné aplikovat při dotazech na grafovou databázi. Umožňují nám efektivně zjistit hledat nejkratší cesty mezi vrcholy, nacházet vrcholy, které pro nás mohou být svým způsobem důležité, nebo třeba analyzovat komunity (skupinky propojených vrcholů) v grafu. Ke grafové databázi Neo4j existuje knihovna s názvem Graph Data Science Library nabízející veškeré algoritmy, které lze po nainstalování pluginu použít. Zkráceně ji budeme označovat gds jako v dotazech Neo4j. V následujících částech si představíme užití pár algoritmů z této knihovny.

stations	stationsCount	totalLength
SCHAERBEEK, BUDA, VILVOORDE, EPPEGEM, WEERDE, MECHELEN	6	18.2057
SCHAERBEEK, VILVOORDE, EPPEGEM, WEERDE, MECHELEN	5	18.2687
SCHAERBEEK, MECHELEN	2	19.8325

Tabulka 4: Výsledná tabulka pro dotaz 4.

5.1 Algoritmy pro hledání cesty v grafu

V praxi je užitečné v grafu najít rychle optimální cestu mezi dvěma, či více vrcholy. Tento problém potřebují řešit například nástroje pracující s grafy reprezentující mapy, které hledají vhodné trasy mezi body na mapě. Dalšími příklady optimalizačních problémů jsou řízení dodavatelských řetězců, hledání nejvhodnějšího autobusového spojení a analýza počítačové sítě.

Pro řešení tohoto problému vznikly různé algoritmy, hledající optimální výsledek. Každý pracuje odlišně a je výhodný pro různé poddruhy problémů. Pár z nich si zde představíme.

5.1.1 Dijkstra

Jeden z nejznámějších algoritmů pro hledání nejkratší cesty. Lze jej aplikovat na neohodnocené grafy, nebo grafy s kladným ohodnocením hran. Jedná se o hladový algoritmus, který k specifikovanému vrcholu vydá nejkratší cestu z tohoto vrcholu do všech ostatních vrcholů. Lze jej také omezit, aby iteroval jen dokud nenajde nejkratší cestu ke konkrétnímu cílovému vrcholu. Dosáhne správného řešení díky pamatování dosud nejkratších nalezených cest do již navštívených vrcholů. V každé iteraci pak vybere jeden vrchol s nejkratší nalezenou cestu ze všech, které dosud nebyly vybrány a následně aktualizuje všechny nejkratší cesty vedoucí přes tento vybraný vrchol. Pokud je nějaký vrchol nedosažitelný nebo ještě nebyl navštívený, má hodnotu ∞ . Nevýhodou tohoto algoritmu je, že pro nalezení nejkratší cesty pro 2 zadané vrcholy je v nejhorším případě potřeba nalézt všechny nejkratší cesty do ostatních vrcholů, než se algoritmus dostane k požadovanému výsledku. To nastává v případě, kdy mezi dvěma vrcholy je nejkratší cesta nejdelší ze všech nejkratších cest z počátečního vrcholu do ostatních vrcholů. Tento problém se pokouší řešit následující algoritmus.

5.1.2 A*

A* (čteme A-star) pracuje podobně jako Dijkstrův algoritmus, ale v mnoha případech ji najde rychleji. Je to protože dostane navíc na vstupu dodatečnou informaci, kterou používá při rozhodování do jakého vrcholu bude algoritmus pokračovat. Toto rozhodnutí je provedeno na základě funkce $f(n) = g(n) + h(n)$, kde funkce $g(n)$ představuje součet ohodnocení nejkratší cesty ze startu přes dosud

navštívené vrcholy do n a funkce $h(n)$ je odhad délky cesty z n do cíle. Funkci $h(n)$ říkáme heuristika. V každé iteraci pak algoritmus pokračuje vrcholem, pro který je hodnota funkce f nejmenší. Pro správné výsledky by heuristika měla být optimistická, tedy odhad by měl být menší nebo roven skutečné vzdálenosti do cíle. Platí, že čím lépe odhadneme heuristiku tím rychleji algoritmus najde řešení. Pro svůj rychlý výpočet je například implementován ve videohrách. Dijkstraův algoritmus je speciálním případem A^* . Pro něj platí $h(v) = 0$ pro všechny vrcholy $v \in V$.

V knihovně gds lze A^* použít například pro vrcholy reprezentující nějakou lokaci, která je přesně určena pomocí zeměpisné šířky a délky. Geografické souřadnice nám slouží jako část heuristické funkce.

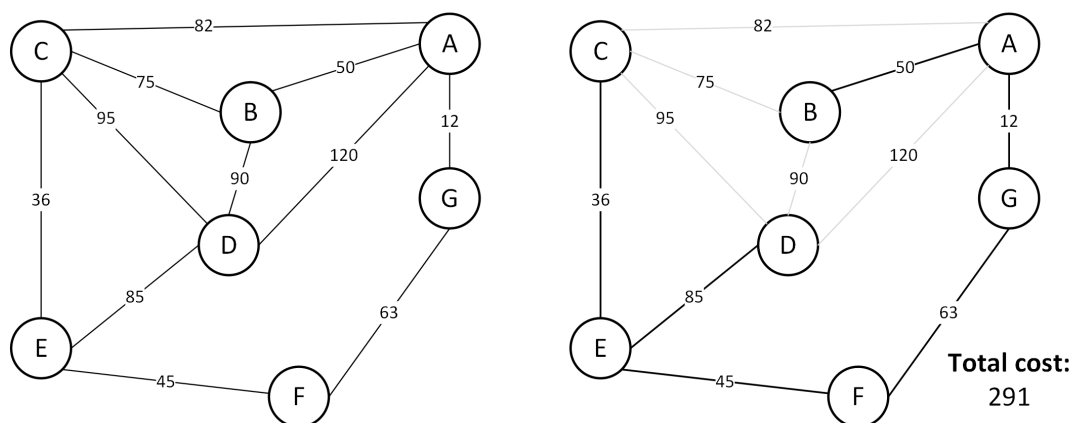
5.1.3 Minimální kostra grafu

Dalším, mezi informatiky dobře známým, problémem je hledání minimální kostry v souvislém grafu. Vstupem je souvislý neorientovaný graf G s ohodnocenými hranami. Výstupem je minimální kostra grafu G . Kostru grafu lze definovat jako podgraf grafu G , který je stromem obsahujícím všechny vrcholy původního grafu. My požadujeme, aby kostra byla minimální, tedy aby součet ohodnocení hran v nalezeném podgrafu byl ze všech možných koster nejmenší. Příklad minimální kostry grafu je na obrázku 13. Problém kostry grafu má více variant. Můžeme též hledat maximální kostru grafu, nebo omezit počet vrcholů v kostře na $k \in \mathbb{N}$ tomuto problému se říká k -spanning tree. Knihovna gds algoritmus na hledání kostry obsahuje. Konkrétně nabízí Jarníkův-Primův algoritmus, poprvé popsáný v roce 1930 českým matematikem Vojtěchem Jarníkem. Jedná se o hladový algoritmus, který vyžaduje na vstupu specifikovat i počáteční vrchol s . Vrátil minimální kostru komponenty vstupního grafu obsahující s . Pokud při implementaci využíváme například binární prioritní frontu máme celkovou složitost $O(|E| \log_2 |V|)$. Tento algoritmus lze využít pro nejlevnější propojení měst vodním potrubím, kde nižší ohodnocení hran znamená menší náklady. Dalším využitím je aproximace problémů patřících do třídy NPTIME, jako je například problém obchodního cestujícího.

5.1.4 Random walk

Jak už název napovídá, Random walk provádí náhodné sledy v grafu specifikované délky. Simuluje průchod grafem přičemž náhodně vybírá hrany. V klasické verzi algoritmu je v každém vrcholu pravděpodobnost vybrání hrany uniformní a nezáleží na předchozích výběrech. Knihovna gds nabízí Random walk druhého řádu. U tohoto algoritmu se specifikují dva parametry:

- *returnFactor* – vyjadřuje tendenci algoritmu se vracet zpět k naposledy navštívenému vrcholu. Hodnoty menší než 1, pravděpodobnost tohoto výběru zvyšují.



Obrázek 13: Algoritmus hledající minimální kostru grafu.

- *inOutFactor* – vyjadřuje tendenci algoritmu zůstat či nezůstat v blízkosti počátečního vrcholu. Čím vyšší hodnota je zadána, tím bude průchod grafem lokálnější.

V klasické verzi jsou oba parametry nastaveny na hodnotu 1. Pravděpodobnost výběru též můžeme ovlivnit pomocí ohodnocených hran. Vlastnost hrany, která je vyšší než jedna pravděpodobnost výběru této hrany zvyšuje. Naopak hodnoty z intervalu $(0, 1)$ pravděpodobnost zmenšují.

Random walk se používá jako součást jiných algoritmů, kde je potřeba množina náhodných, ale navzájem propojených vrcholů. V následující ukázce byl algoritmus `gds.randomWalk` použit na železniční stanice, které mají vlastnost `stationId` menší než 10. To jsou stanice s vlastností `stationName` AALST, AALTER a AARSCHOT. Tabulka 5 zobrazuje výsledky s parametrem `inOutFactor` rovným 2. V tabulce 6 jsou vidět výsledky s parametrem `inOutFactor` rovným hodnotě 0.2.

stations
AALST, EREMBODEGEM, DENDERLEEUEW, IDDERGEM, DENDERLEEUEW, GENT-ST-P, DENDERLEEUEW, GENT-ST-P
AALTER, MARIA-AALTER, AALTER, BELLEM, HANSBEKE, BELLEM, HANSBEKE, BELLEM
AARSCHOT, BEGIJNENDIJK, AARSCHOT, LANGDORP, AARSCHOT, "BEGIJNENDIJK, AARSCHOT, LANGDORP

Tabulka 5: Výsledek algoritmu Random walk s parametrem `inOutFactor` nastaveným na hodnotu 2 zavolaným na stanice s vlastností `stationId` menší než 10.

5.2 Algoritmy pro míru centrality

V rámci grafu můžeme identifikovat důležitost vrcholů pomocí algoritmů centrality. Můžeme detekovat jedince, které jsou svým způsobem v grafu významější

stations
AALST, LEDE, AALST, EREMBODEGEM, DENDERLEEUEW, MELLE, KWATRECHT, WETTEREN
AALTER, BELLEM, HANSBEKE, LANDEGEM, HANSBEKE, BELLEM, AALTER, MARIA-AALTER
AARSCHOT, LANGDORP, TESTELT, LANGDORP, BEGIJNENDIJK, BOOISCHOT, HEIST-BERG, MELKOUWEN

Tabulka 6: Výsledek algoritmu Random walk s parametrem inOutFactor nastaveným na hodnotu 0.2 zvolaným na stanice s vlastností stationId menší než 10.

než ostatní. Tyto algoritmy se dají využít například k hledání kritických částí v telekomunikačních nebo počítačových sítích, tedy těch přes které proudí spousta informací a jejichž výpadek by představoval velké potíže. Hledání důležitých lidí v rámci organizace. Hledání odlehklých dat, které mohou odhalit, že dochází k podvodům a podobně. Knihovna gds nabízí spoustu algoritmů pro měření centrality. My se podíváme na pár z nich.

5.2.1 Degree Centrality

Nejznámějším a nejjednodušší mírou centrality je degree centrality. Každému vrcholu v síti přiřazuje skóre podle stupně daného vrcholu. V sociálních sítích mívají vrcholy i stupně v řádu desítek milionů (jedná se o vrcholy, reprezentující účty známých osobností nebo organizací). Těmto vrcholům říkáme super-uzly. Abychom prošli všechny sousedy super-uzlu potřebujeme značně více času, než v případě obyčejného vrcholu. To se negativně projevuje v časové odezvě u dotazů pracujících s takovými vrcholy. Degree centrality nám umožňuje takové vrcholy odhalit. Vrcholy s větším stupněm také lze obecně považovat za důležitější, neboť přes ně může proudit více informací díky velkému počtu hran. Příkladem může být graf reprezentující železniční síť. Čím více máme z dané železniční stanice spojení, tím spíše se jedná o významější stanici.

5.2.2 Closeness Centrality

Vrcholy lze dále hodnotit podle closeness centrality. Vrchol s nejvyšším skóre je ten, pro který je součet nejkratších cest do všech ostatních vrcholů co nejmenší. Skóre vrcholu $u \in V$ v grafu $G = \langle V, E \rangle$ se pak počítá podle vzorce

$$closenessCentrality(u) = \frac{1}{\sum_{v \in V} dist(u, v)}, \quad (1)$$

nebo se normalizuje

$$normalizedClosenessCentrality(u) = \frac{|V| - 1}{\sum_{v \in V} dist(u, v)}. \quad (2)$$

V případě, že graf není souvislý, lze použít harmonic centrality, které se elegantně vypořádá s nedosažitelnými vrcholy. Tento a následující algoritmus mohou být v případě velkých grafů výpočetně dost náročné a tedy se často přistupuje k ne zcela přesným, zato mnohonásobně rychlejším, algoritmům.

5.2.3 Betweenness Centrality

Algoritmus spočítá nejkratší cesty mezi všemi možnými páry vrcholů. Následně každý vrchol obdrží skóre, které je tím vyšší, čím více nejkratších cest vede přes daný vrchol. Tyto hodnoty mohou pro nás být klíčové například v grafu, kde vrcholy představují křižovatky a hrany cesty mezi nimi. Informaci, kterými křižovatkami prochází nejvíce nejkratších cest, můžeme využít pro odhad těch nejvíce zatížených křižovatek. Případně díky ní můžeme odhadnout, kde se pravděpodobně bude pohybovat nejvíce lidí a tedy je potřeba zvýšit bezpečnost.

5.2.4 PageRank

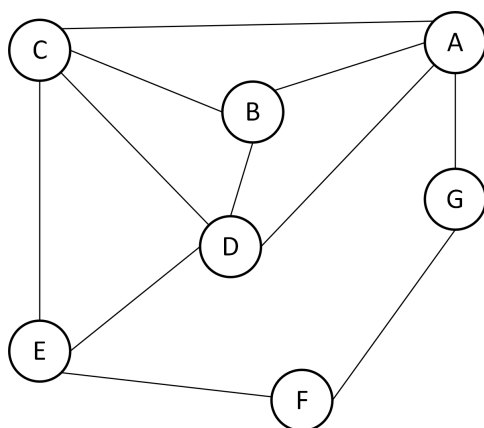
S tímto řešením přišla na trh společnost Google. Byla určena pro hodnocení významnosti webových stránek. Webové stránky si lze představit jako vrcholy grafu. Hrany mezi nimi reprezentují odkazy na stránkách odkazující na jiné stránky. Důležitost vrcholu je určena nejen důležitostí vrcholu samotného, ale také důležitostí vrcholů na tuto stránku odkazujících. Vysoké skóre vrcholu tedy znamená, že je vrchol propojen s vrcholy, které mají též vysoké skóre. Toto řešení je založeno na lineární algebře, speciálně na vlastních vektorech a stochastických maticích. Výpočet skóre většinou vede na nekonečný výpočet a proto se omezuje počet iterací tohoto výpočtu, nebo se udává maximální možná odchylka. PageRank se používá na orientované grafy. Pro neorientované grafy lze použít algoritmus eigenvector centrality.

5.2.5 Použití algoritmů pro míru centrality

Výsledky aplikace jednotlivých algoritmů pro měření centrality na graf z obrázku 14 je znázorněn v tabulce 7.

Vrchol	A	B	C	D	E	F	G
Degree Centrality	4	3	4	4	3	2	2
Closeness Centrality	0.75	0.6	0.75	0.75	0.67	0.55	0.6
Betweenness Centrality	3.33	0.0	1.33	1.33	2.67	1.0	1.33
Eigen Vector Centrality	0.45	0.41	0.49	0.49	0.32	0.14	0.17

Tabulka 7: Algoritmy pro měření centrality aplikované na jednotlivé vrcholy z grafu na obrázku 14.



Obrázek 14: Příklad grafu, na kterém jsou prováděny algoritmy.

5.3 Detekce komunit

Tyto algoritmy lze využít pro hledání určitých skupin vrcholů, které mají mezi sebou více hran, než mají s ostatními vrcholy grafu. Odhalením těchto shluků vrcholů můžeme zkoumat strukturu našich dat z jiného úhlu pohledu, nebo třeba odhadovat podobné chování a preference členů v rámci skupiny. Příklad lze nalézt v sociálních sítích, kde můžeme sledovat komunity blízkých přátel, které tvoří shluky propojených vrcholů. Znalost komunit lze též využít pro doporučování produktů uživatelům s podobným nákupním chováním. Algoritmy pro detekci komunit se dělí na deterministické a nedeterministické. Deterministické vrací pro stejný graf vždy stejný výsledek, zatímco výsledek u nedeterministických může být pro stejný graf mírně odlišný. Mezi deterministické se řadí počítání trojúhelníků, výpočet shlukovacího koeficientu, nebo hledání komponent. Mezi nedeterministické patří například Label Propagation nebo Louvain Modularity. Stejně jako u míry centrality si zde ukážeme pouze pár algoritmů.

5.3.1 Počítání trojúhelníků

Algoritmus pro každý vrchol určí počet trojúhelníků, kterých je daný vrchol součástí. Trojúhelník je zde definován jako skupina tří navzájem propojených vrcholů. Trojúhelníky nám mohou určit do jaké míry jsou vrcholy v našem grafu propojeny. Čím je graf hustší, tím více trojúhelníků se v něm nachází. Algoritmus je oblíbený při analýze sociálních sítí, kde pomáhá detekovat komunity, nebo zkoumání skupin webových stránek s podobným námětem podle odkazů mezi nimi. Počet trojúhelníků lze využít též pro výpočet shlukovacího koeficientu vrcholu, který určuje pravděpodobnost, s jakou jsou jeho sousedé propojeni (tvoří s ním trojúhelník). Shlukovací koeficient vrcholu u je pak určen vztahem

$$\text{ClusteringCoefficient}(u) = \frac{2 * \text{TriangleCount}(u)}{\text{deg}(u) * (\text{deg}(u) - 1)}. \quad (3)$$

Pro ukázkový graf na obrázku 14 počet trojúhelníků a shlukovací koeficient

pro jednotlivé vrcholy zobrazuje tabulka 8. Vidíme, že vrchol A je součástí tří trojúhelníků, které jsou tvořeny vrcholy A-B-C, A-B-D a A-C-D. Stupeň vrcholu je 4. Dosazením do vzorce 3 dostáváme shlukovací koeficient 0.5.

Vrchol	A	B	C	D	E	F	G
Triangle Count	3	3	4	4	1	0	0
Clustering Coefficient	0.5	1.0	0.67	0.67	0.33	0.0	0.0

Tabulka 8: Počet trojúhelníků a shlukovací koeficient jednotlivých vrcholů z grafu na obrázku 14.

5.3.2 Hledání komponent

Komponentu v grafu tvoří skupina vrcholů, které jsou propojeny a lze se z jednoho vrcholu dostat do dalšího. Najít těchto komponent je užitečné pro pochopení grafové struktury. Algoritmy řešící tento problém jsou rychlé a je možné je využít v grafu, který se často aktualizuje, pro detekci nových vrcholů mezi skupinami. Této znalosti lze využít při různých analýzách. Existuje i varianta, hledající silně souvislé komponenty. Pro graf na obrázku 14 nám algoritmus wcc z knihovny gds vrátí, že všechny vrcholy grafu jsou součástí jedné komponenty.

Závěr

Závěr práce v „českém“ jazyce. V budoucnu je možné přidat entitu cestovní kanceláře, která bude pořádat cesty do destinací a uživatelé po objednání budou do této cesty připojeni. To otevírá dveře dalším entitám například pro hodnocení cestovních kanceláří, nebo vztahům které se týká zaměstnanosti uživatelů u dané cestovky.

Conclusions

Thesis conclusions in “English”.

